.NET Senior

Interview Questions & Answers

215 Real-World Questions with Clear Answers

C# & .NET | ASP.NET Core | EF Core | Security | System Design

2025 Edition

Table of Contents

BUCKET 1 — C# AND .NET

Ch 1 — Types and Type System

Q1 What is the difference between a value type and a reference type in C#?

Answer

Value types live on the stack (or inline inside an object). Copying one gives you a completely independent copy — changing the copy leaves the original untouched.

Reference types live on the heap. Assigning one to another variable just copies the pointer, so both variables point at the same object. Mutating through one is visible through the other.

Common value types: int, bool, double, struct, enum. Common reference types: class, string, array, delegate.

Example

```
int a = 5;
int b = a; // independent copy
b = 10;
Console.WriteLine(a); // still 5

var p1 = new Point { X = 1 };
var p2 = p1; // same heap object
p2.X = 99;
Console.WriteLine(p1.X); // 99
```

Q2 What is the difference between float, double, and decimal?

Answer

float (32-bit) and double (64-bit) are binary floating-point — fast but imprecise for exact decimal fractions. decimal (128-bit) uses base-10, so 0.1 + 0.2 is exactly 0.3.

Rule of thumb: scientific/graphics work use double. Money and financial figures use decimal. float is rarely needed except for high-volume GPU work.

Example

```
double d = 0.1 + 0.2;
Console.WriteLine(d); // 0.30000000000000004

decimal m = 0.1m + 0.2m;
Console.WriteLine(m); // 0.3 (exact)
```

Q3 What is the difference between string and StringBuilder?

Answer

string is immutable — every concatenation creates a new heap object. For a few literals that is fine; inside a loop with thousands of appends it causes a heap explosion.

StringBuilder keeps one mutable internal buffer and only allocates when it fills up. Call ToString() once at the end.

Example

```
// Bad in a tight loop
string s = "";
for (int i = 0; i < 10_000; i++) s += i; // 10,000 allocs

// Good
var sb = new StringBuilder();
for (int i = 0; i < 10_000; i++) sb.Append(i);
string result = sb.ToString(); // 1 alloc
```

Q4 What is boxing and unboxing and what does it cost?

Answer

Boxing wraps a value type in a heap object so it can be passed as object. Unboxing casts it back. Each operation involves a heap allocation plus a copy — expensive on hot paths.

Classic trap: ArrayList boxes every int you put in. Use List<int> instead.

Example

```
int x = 42;
object boxed = x; // boxing — heap alloc
int unboxed = (int)boxed; // unboxing

var list = new ArrayList();
list.Add(42); // boxes — use List<int> instead
```

Q5 What is a ref struct and what restrictions does it have?

Answer

A ref struct is guaranteed to stay on the stack — the compiler prevents it escaping to the heap. That makes it safe to wrap a raw memory pointer, which is exactly what Span<T> does.

Restrictions: can't be boxed, can't be a generic type argument, can't be a field on a regular class, can't be used in async methods or capturing lambdas.

Example

```
ref struct StackBuffer
{
public Span<byte> Data;
}
// Span<T> is itself a ref struct — that's why you can't
// store a Span<T> as a field on a regular class.
```

Ch 2 — Operators, Statements, Expressions

Q6 What is the null-coalescing operator (??) and null-conditional operator (?.) ?

Answer

?? returns the left side if it isn't null, otherwise the right side — a safe default.

?. short-circuits the entire member-access chain the moment it hits null, returning null instead of throwing NullReferenceException. Combine them for clean null-safe one-liners.

Example

```
string name = null;
string display = name ?? "Guest"; // "Guest"
int? len = name?.Length; // null, no exception

Console.WriteLine(name?.ToUpper() ?? "No name"); // "No name"
```

Q7 What does a foreach loop compile to under the hood?

Answer

The compiler rewrites foreach into a while loop that calls GetEnumerator(), then loops calling MoveNext() and reading Current. If the enumerator implements IDisposable, a try/finally disposes it even if the body throws.

Example

```
// foreach (var item in list) { ... }
// compiles roughly to:
```

```
var e = list.GetEnumerator();
try {
while (e.MoveNext()) {
var item = e.Current;
// body
}
} finally { e.Dispose(); }
```

Ch 3 — OOP: Classes, Inheritance, Polymorphism

Q8 What is the difference between an abstract class and an interface?

Answer

An abstract class can hold state (fields), constructors, and fully implemented methods. A class can only inherit from one.

An interface since C# 8 can have default method bodies but no instance state. A class can implement any number of interfaces.

Use abstract class for shared logic to inherit. Use interface for a capability contract.

Example

```
abstract class Shape {
public abstract double Area();
public void Print() => Console.WriteLine(Area());
}
interface IResizable { void Resize(double factor); }

class Circle : Shape, IResizable {
public double Radius { get; private set; }
public Circle(double r) => Radius = r;
public override double Area() => Math.PI * Radius * Radius;
public void Resize(double f) => Radius *= f;
}
```

Q9 What is polymorphism and how does C# support it via virtual/override?

Answer

Polymorphism means one variable behaves differently at runtime depending on the actual type of the object it holds. Mark the base method virtual, override it in a subclass, and the runtime dispatches via the vtable — no if/switch on the caller's side.

Example

```
class Animal {
public virtual string Sound() => "...";
}
class Dog : Animal {
public override string Sound() => "Woof";
}
Animal a = new Dog();
Console.WriteLine(a.Sound()); // "Woof" — runtime dispatch
```

Q10 What is a primary constructor (C# 12)?

Answer

A primary constructor puts parameters directly on the class or struct declaration. Those parameters are in scope everywhere in the type body — no separate constructor block needed.

Example

```
class Point(double x, double y)
{
public double X => x;
public double Y => y;
public double Distance() => Math.Sqrt(x*x + y*y);
}
```

Q11 What are init-only setters and what problem do they solve?

Answer

init setters let you assign a property during object initialiser syntax but block any further change afterwards. This gives immutability without forcing everything through a constructor — ideal for DTOs.

Example

```
class Order {
public int Id { get; init; }
public string Product { get; init; }
}
var o = new Order { Id = 1, Product = "Book" }; // ok
o.Id = 2; // compile error — init-only
```

Q12 How does the runtime dispatch a virtual call vs. an interface call?

Answer

Virtual call: the object header points to its Method Table. The vtable slot for that method is at a fixed offset — one pointer indirection, very fast.

Interface call: the runtime looks up the interface map inside the Method Table to find the slot — slightly more work, though the JIT often inlines or caches it.

Example

```
Animal a = new Dog();
a.Sound(); // virtual — one vtable slot lookup (fast)

IResizable r = new Circle(5);
r.Resize(2); // interface — interface map lookup (slightly slower)
```

Ch 4 — Records, Anonymous Types, Tuples

Q13 What is a record in C# and how does it differ from a class?

Answer

A record is a reference type with compiler-generated value semantics: Equals, GetHashCode, == and != all compare by property values rather than object identity. You also get a with-expression for non-destructive mutation.

Example

```
record Person(string Name, int Age);

var p1 = new Person("Alice", 30);
var p2 = p1 with { Age = 31 }; // new object, Name copied
Console.WriteLine(p1 == p2); // False
Console.WriteLine(p1.Name == p2.Name); // True
```

Q14 How does equality work for records compared to classes?

Answer

Classes compare by identity by default — two variables are equal only if they point to the exact same heap object. Records compare by value — two records with the same property values are equal even if they are different instances.

Example

```
var a = new Person("Bob", 25);
var b = new Person("Bob", 25);
Console.WriteLine(a == b); // True for record, False for class
```

Q15 When would you prefer a record struct over a record class?

Answer

When the object is small, short-lived, and in a hot path. record struct allocates on the stack — zero GC pressure. Coordinate pairs, colour values, and price ranges are good candidates. You still get value equality and with-expressions.

Example

```
record struct Point(double X, double Y);

var p = new Point(1, 2);
var shifted = p with { X = 5 }; // new stack value
```

Ch 5 — Generics and Variance

Q16 What are generic constraints and what kinds are available?

Answer

Constraints tell the compiler what operations are legal on a type parameter.

where T : class -- reference type where T : struct -- value type where T : new() -- has parameterless constructor where T : SomeBase -- inherits SomeBase where T : IFoo -- implements IFoo where T : notnull -- non-nullable

Example

```
T CreateAndLog<T>(ILogger log)
where T : class, new()
{
var obj = new T();
log.LogInformation(obj.ToString());
return obj;
}
```

Q17 Explain covariance and contravariance with in and out.

Answer

out (covariance): type only flows OUT. IEnumerable<Dog> is assignable to IEnumerable<Animal> because you only read from it.

in (contravariance): type only flows IN. Action<Animal> is assignable to Action<Dog> because a handler that accepts any Animal can certainly handle a Dog.

Example

```
IEnumerable<Dog> dogs = new List<Dog>();
IEnumerable<Animal> animals = dogs; // covariant -- ok

Action<Animal> feedAnimal = a => Console.WriteLine("fed");
Action<Dog> feedDog = feedAnimal; // contravariant -- ok
```

Q18 How does the JIT specialise generic code for value types vs. reference types?

Answer

For reference types the JIT generates ONE shared native code path (all references are the same size). For value types it generates a separate native code path per type — so List<int> and List<double> get different machine code, avoiding boxing entirely.

Example

```
List<int> ints = new(); ints.Add(42); // no boxing
List<object> objs = new(); objs.Add(42); // boxing — int wrapped in object
```

Ch 6 — Collections

Q19 What is the difference between IEnumerable<T> and ICollection<T>?

Answer

IEnumerable<T> is the bare minimum: forward-only iteration. ICollection<T> adds Count, Add, Remove, Contains, and Clear. Use IEnumerable when you only need to loop; ICollection when the caller needs to modify or measure.

Example

```
void PrintAll(IEnumerable<int> items) {
foreach (var x in items) Console.WriteLine(x);
}
void AddIfMissing(ICollection<int> items, int val) {
if (!items.Contains(val)) items.Add(val);
}
```

Q20 What are the concurrent collections in .NET?

Answer

ConcurrentDictionary -- thread-safe key-value store with atomic AddOrUpdate/GetOrAdd. ConcurrentQueue -- lock-free FIFO, great for producer-consumer. ConcurrentBag -- unordered, best when adder and remover are the same thread. BlockingCollection -- wraps any concurrent collection, adds blocking Take() with optional capacity.

Example

```
var dict = new ConcurrentDictionary<string, int>();
dict.AddOrUpdate("hits", 1, (k, old) => old + 1);

var q = new ConcurrentQueue<string>();
q.Enqueue("job1");
q.TryDequeue(out var job);
```

Q21 What is FrozenDictionary and when does it beat a regular dictionary?

Answer

FrozenDictionary is a read-only dictionary optimised at construction time. Because it never changes, the runtime can pick a perfect hash layout. For read-heavy lookup tables like routing tables or config maps — built once, read millions of times — it is measurably faster.

Example

```
var config = new Dictionary<string,string> {
["host"] = "localhost", ["port"] = "5432"
}.ToFrozenDictionary();

var host = config["host"]; // fast repeated reads
```

Q22 How is Dictionary<TKey,TValue> implemented internally?

Answer

It uses an array of buckets. The key's hash code selects a bucket. Collisions are handled by chaining. Average lookup is O(1). A poor GetHashCode that returns the same value for many keys degrades to O(n) linear scan.

Example

```
record Point(int X, int Y)
{
// Good spread prevents O(n) degradation
public override int GetHashCode() =>
HashCode.Combine(X, Y);
}
```

Q23 How does ConcurrentDictionary handle concurrent updates?

Answer

It splits the bucket array into stripes and assigns one lock per stripe. Reads are mostly lock-free via volatile reads. Writes lock only the stripe that the key hashes into, so two threads writing to different keys rarely contend.

Example

```
var cd = new ConcurrentDictionary<int, int>();
// Thread-safe increment — no external lock needed
cd.AddOrUpdate(1, addValue: 1,
updateValueFactory: (k, v) => v + 1);
```

Ch 7 — Delegates, Events, Lambdas

Q24 What is the difference between a delegate and an event?

Answer

A delegate is a type-safe function pointer — anyone holding it can invoke it directly. An event wraps that delegate and restricts outside code to only += and -=. Only the declaring class can fire it. Events enforce the publisher/subscriber pattern without letting subscribers call each other.

Example

```
class Button {
public event EventHandler Clicked;
public void SimulateClick() => Clicked?.Invoke(this, EventArgs.Empty);
}
// Outside: btn.Clicked += handler; -- ok
// Outside: btn.Clicked(this, e); -- compile error
```

Q25 What is a closure and what are the captured-variable rules?

Answer

A closure is a lambda that captures a variable from its enclosing scope. The compiler creates a hidden class to hold it. That variable is shared — changes inside the lambda are visible outside. Classic bug: capturing a loop variable directly means all lambdas share the same final value.

Example

```
var funcs = new List<Func<int>>();
for (int i = 0; i < 3; i++) {
int copy = i; // capture a fresh copy each iteration
funcs.Add(() => copy);
}
// Without copy: all three lambdas return 3 (loop's final i)
```

Q26 What is an expression tree and how does it differ from a delegate?

Answer

A delegate is compiled native code you can call but not inspect. An expression tree is an object graph describing the code — you can inspect every node, translate it (EF Core translates LINQ trees to SQL), or compile it to a delegate at runtime.

Example

```
Expression<Func<int, bool>> expr = x => x > 5;

var binary = (BinaryExpression)expr.Body;
Console.WriteLine(binary.NodeType); // GreaterThan

var func = expr.Compile();
Console.WriteLine(func(10)); // True
```

Q27 What is a multicast delegate and what happens when one handler throws?

Answer

A multicast delegate holds a list of methods and calls them in registration order. If one handler throws the exception propagates immediately and remaining handlers never run. To guarantee all handlers run, iterate GetInvocationList() manually with try/catch per call.

Example

```
Action notify = () => Console.WriteLine("A");
notify += () => throw new Exception("B failed");
notify += () => Console.WriteLine("C"); // never runs if B throws

// Safe:
foreach (Action h in notify.GetInvocationList())
try { h(); } catch { /* log */ }
```

Ch 8 — LINQ

Q28 What is the difference between IEnumerable<T> and IQueryable<T> in LINQ?

Answer

IEnumerable runs the query in memory — data is fetched first, filtered in C#. IQueryable holds an expression tree; the provider (EF Core) translates the whole query to SQL before any data crosses the wire. Always use IQueryable against a database.

Example

```
// IQueryable -- translated to SQL, only matching rows fetched
var adults = db.Users.Where(u => u.Age > 18).ToList();

// IEnumerable -- loads ALL rows first, then filters in C#
var adults2 = db.Users.AsEnumerable().Where(u => u.Age > 18).ToList();
```

Q29 What is deferred execution in LINQ?

Answer

A LINQ query doesn't run when you write it — it runs the moment you iterate it (ToList, First, foreach, etc.). Changes to the source made after the query is written but before iteration are visible. Running the query twice hits the source twice.

Example

```
var nums = new List<int> { 1, 2, 3 };
var q = nums.Where(n => n > 1); // nothing runs yet
```

```
nums.Add(4);
foreach (var n in q) // executes NOW -- sees 4
Console.Write(n + " "); // 2 3 4
```

Q30 What is IAsyncEnumerable<T> and how do you consume it?

Answer

IAsyncEnumerable<T> streams items one at a time from an async source without buffering everything in memory first — think reading rows page by page. Consume with await foreach.

Example

```
async IAsyncEnumerable<int> GetNumbers() {
for (int i = 0; i < 5; i++) {
await Task.Delay(100);
yield return i;
}
}
await foreach (var n in GetNumbers())
Console.WriteLine(n); // arrives one at a time
```

Q31 How does the EF Core LINQ provider translate expression trees to SQL?

Answer

When you query a DbSet, EF Core receives an IQueryable with an expression tree. The query provider walks the tree, maps C# constructs to SQL, and sends one SQL statement to the database. No filtering happens in C#.

Example

```
var orders = await db.Orders
.Where(o => o.Total > 100 && o.Status == "Paid")
.OrderBy(o => o.CreatedAt)
.ToListAsync();
// SELECT * FROM Orders WHERE Total > 100 AND Status='Paid'
// ORDER BY CreatedAt
```

Q32 Why is Count() == 0 worse than Any() on an IEnumerable?

Answer

Count() scans every element to get the total. Any() stops at the first — O(1) for the question 'is there anything here?'. On a million-item sequence Any() returns immediately; Count() scans the lot. Exception: List<T>.Count and array.Length are O(1) properties.

Example

```
var items = GetMillionItems();
bool slow = items.Count() > 0; // scans everything
bool fast = items.Any(); // stops at first element
```

Ch 9 — Pattern Matching

Q33 What is a list pattern (C# 11)?

Answer

List patterns let you match the shape and content of an array or list in a switch expression. Pin specific positions, use _ as a single-element wildcard, and .. for any-length slices.

Example

```
int[] nums = { 1, 2, 3, 4 };
var desc = nums switch {
```

```
[1, 2, ..] => "starts with 1,2",
[.., 4] => "ends with 4",
[] => "empty",
_ => "other"
}; // "starts with 1,2"
```

Ch 10 — Nullable Reference Types

Q34 Is the nullable reference type annotation enforced at runtime?

Answer

No — it is purely a compile-time hint. At runtime string? and string are identical types. The annotations let the compiler warn about potential null dereferences but insert no runtime checks. You still need explicit null guards for runtime safety.

Example

```
#nullable enable
string? name = null;
Console.WriteLine(name.Length); // compiler warning, but compiles
// -> NullReferenceException at runtime

if (name is null) throw new ArgumentNullException(nameof(name));
```

Ch 11 — C# Error Handling

Q35 What is the difference between throw ex and throw inside a catch block?

Answer

throw ex resets the stack trace — you lose where the exception actually originated. Bare throw re-throws with the original stack trace intact. Always use bare throw when you just want to rethrow after logging.

Example

```
catch (Exception ex)
{
_log.LogError(ex, "failed");
throw; // correct -- original stack trace kept
// throw ex; // wrong -- stack trace reset to this line
}
```

Q36 What is the cost of throwing exceptions in .NET?

Answer

Throwing is expensive: the runtime walks every stack frame to build the trace, allocates the exception object, and unwinds the stack. On a hot path this is thousands of times slower than returning a result. Use exceptions for truly unexpected failure, not control flow.

Example

```
// Bad -- exception for ordinary control flow
try { user = FindUser(id); }
catch (NotFoundException) { user = null; }

// Good -- return null / Result type instead
user = FindUserOrDefault(id);
```

Q37 How does the CLR unwind the stack when an exception is thrown?

Answer

The CLR scans exception-handling tables baked into each JIT method, walks up the call stack for a matching catch, and runs all finally blocks along the way. Zero-cost tables mean no overhead unless an exception is actually thrown.

Example

```
void A() { B(); }
void B() { throw new Exception("boom"); }

// Unwind: B checks -- no catch.
// A checks -- no catch.
// Keeps going until match or app crash.
```

Ch 12 — Async/Await and Tasks

Q38 What is the difference between Task and ValueTask?

Answer

Task always allocates a heap object. ValueTask is a struct — when the result is already ready (cache hit) it can return with zero allocation. Use ValueTask on methods that are frequently synchronous. Rule: never await a ValueTask more than once.

Example

```
ValueTask<int> GetCachedAsync(int key)
{
if (_cache.TryGetValue(key, out var v))
return new ValueTask<int>(v); // zero alloc
return new ValueTask<int>(LoadFromDbAsync(key));
}
```

Q39 What is ConfigureAwait(false) and when should you use it?

Answer

By default await resumes on the captured SynchronizationContext (e.g., UI thread). ConfigureAwait(false) says resume on any thread-pool thread. In library code this is a small win and prevents certain deadlocks. In UI event handlers where you need to touch controls after the await, don't use it.

Example

```
public async Task<string> FetchDataAsync(string url)
{
using var client = new HttpClient();
return await client.GetStringAsync(url)
.ConfigureAwait(false);
}
```

Q40 What state machine does the compiler generate for an async method?

Answer

The compiler rewrites every async method into a struct implementing IAsyncStateMachine. Each await point becomes a numbered state. When an await suspends, locals are saved and the method returns. When the awaited task completes MoveNext() resumes from the saved state.

Example

```
// You write:
async Task<int> AddAsync(int a, int b) {
await Task.Delay(1);
return a + b;
}
// Compiler generates a hidden struct:
// State 0 -> start delay, save a+b, return
```

```
// State 1 -> resume, compute a+b, complete task
```

Q41 What is SynchronizationContext and how does it affect await resumption?

Answer

SynchronizationContext controls how continuations are scheduled back to a specific thread. ASP.NET Core has no SynchronizationContext, so await resumes on a free thread-pool thread.

Important: even in ASP.NET Core, blocking with .GetResult() or .Wait() wastes a thread-pool thread and can cause starvation under load. Never do it in production.

Example

```
// ASP.NET Core -- no SynchronizationContext.
// The classic UI-thread deadlock can't occur,
// but blocking STILL starves the thread pool:
var result = SomeAsync().GetAwaiter().GetResult(); // avoid

// In WinForms/WPF -- .Result WILL deadlock!
```

Q42 What is the difference between async void and async Task?

Answer

async Task lets callers await and catch exceptions. async void cannot be awaited — exceptions thrown inside crash the process. The only accepted use of async void is event handlers where the signature is forced on you.

Example

```
async void DoWork() { throw new Exception(); } // crashes process
async Task DoWorkAsync() { throw new Exception(); } // caller can catch

// Event handler -- signature forced, acceptable:
btn.Click += async (s, e) => { await LoadAsync(); };
```

Q43 Why does calling .Result on a Task cause a deadlock in some contexts?

Answer

The async method schedules its continuation to resume on the current SynchronizationContext thread. But .Result blocks that same thread waiting for the Task. The continuation can never run — deadlock.

Example

```
// WinForms button click:
private void btn_Click(object s, EventArgs e)
{
var data = FetchDataAsync().Result; // deadlock!
// FetchDataAsync wants the UI thread back,
// but the UI thread is blocked right here.
}
// Fix: make the handler async and await
```

Ch 13 — Threading and Synchronization

Q44 What is lock and what does it compile to?

Answer

lock is syntax sugar for Monitor.Enter and Monitor.Exit wrapped in try/finally, ensuring the lock is always released. Always lock on a private readonly object — never on this or a public type.

Example

```
private readonly object _sync = new();
lock (_sync) { _count++; }

// Compiles to:
bool taken = false;
Monitor.Enter(_sync, ref taken);
try { _count++; }
finally { if (taken) Monitor.Exit(_sync); }
```

Q45 What is the new Lock type in C# 13?

Answer

System.Threading.Lock is a dedicated lock type. The compiler recognises it and emits more optimised primitives than generic Monitor.Enter/Exit. You still use the same lock keyword.

Example

```
private readonly Lock _lock = new();
lock (_lock) // compiler emits optimised EnterScope
{
_count++;
}
```

Q46 What is Channel<T> and how does it compare to BlockingCollection?

Answer

Channel<T> is async-first — both sides use WriteAsync/ReadAsync, no thread is ever blocked. BlockingCollection blocks a real thread while waiting, wasting thread-pool resources. In ASP.NET Core background services always prefer Channel<T>.

Example

```
var ch = Channel.CreateUnbounded<string>();

// Producer
await ch.Writer.WriteAsync("hello");

// Consumer
await foreach (var msg in ch.Reader.ReadAllAsync())
Console.WriteLine(msg);
```

Q47 What happens when the thread pool gets starved?

Answer

When too many threads are blocked (usually from .Result on async code), the pool queue fills but has no free threads to drain it. The pool injects new threads slowly — one per second — causing latency spikes. Diagnose with dotnet-counters watching ThreadPool.Queue length.

Example

```
// Classic cause: blocking an async call
public IActionResult Get()
{
var result = _service.GetDataAsync().Result; // blocks thread
return Ok(result);
}
// Fix: make the action async and await
```

Ch 14 — Memory Management and GC

Q48 Explain GC generations 0, 1, and 2.

Answer

Gen 0 is collected most often and holds new short-lived objects. Survivors are promoted to Gen 1, then to Gen 2 which is collected rarely and holds long-lived objects like static caches. The insight: most objects die young, so collecting Gen 0 alone clears most garbage cheaply.

Example

```
var temp = new byte[100]; // short-lived, Gen 0

// Long-lived -- eventually reaches Gen 2
private static readonly Dictionary<string, Config> _cache = new();
```

Q49 What is the Large Object Heap (LOH) and why does it matter?

Answer

Objects >= 85,000 bytes go straight to the LOH. The LOH is only collected during Gen 2 collections (rare) and isn't compacted by default, so it fragments. Fix: rent large buffers from ArrayPool instead of allocating fresh each time.

Example

```
// Bad -- 100 KB on the LOH every call
byte[] buf = new byte[100_000];

// Good -- borrow from pool, no LOH pressure
byte[] buf = ArrayPool<byte>.Shared.Rent(100_000);
try { /* use buf */ }
finally { ArrayPool<byte>.Shared.Return(buf); }
```

Q50 What is the difference between workstation GC and server GC?

Answer

Workstation GC (default on desktop) runs on the application thread, keeping pauses short. Server GC (default in ASP.NET Core) creates a separate heap and GC thread per logical CPU, giving much higher throughput on multi-core machines at the cost of more memory.

Example

```
<!-- .csproj -->
<PropertyGroup>
<ServerGarbageCollection>true</ServerGarbageCollection>
</PropertyGroup>
```

Q51 What is the dispose pattern and why does Dispose(bool) have a parameter?

Answer

The parameter separates managed cleanup (run only from Dispose) from unmanaged cleanup (run from both Dispose and finalizer). When true you were called from Dispose() — safe to release managed objects too. When false you were called from the finalizer — only release unmanaged handles.

Example

```
class ResourceHolder : IDisposable {
private bool _disposed;
protected virtual void Dispose(bool disposing) {
if (_disposed) return;
if (disposing) _managed?.Dispose(); // managed
_nativeHandle.Free(); // unmanaged
_disposed = true;
}
public void Dispose() { Dispose(true); GC.SuppressFinalize(this); }
~ResourceHolder() { Dispose(false); }
```

```
}
```

Q52 How do you diagnose a memory leak in a .NET application?

Answer

1. Watch process memory with dotnet-counters or Prometheus. 2. Capture a dump with dotnet-dump or ProcDump when memory is high. 3. Analyse with dotnet-dump analyze, Visual Studio, or WinDbg+sos. 4. Run dumpheap -stat to spot types with unexpectedly high instance counts.

Common culprits: static collections growing forever, event subscriptions never removed, IDisposable not disposed.

Example

```
// Classic leak: subscription never removed
Publisher pub = GetPublisher();
pub.OnData += Handler; // pub holds a reference back
// Fix: pub.OnData -= Handler; when done
```

Ch 15 — Reflection and Dynamic

Q53 What is the performance cost of reflection and how do you reduce it?

Answer

Reflection bypasses JIT optimisations and does dynamic dispatch — 10-100x slower than direct calls. Cache MethodInfo/PropertyInfo at startup so you pay the lookup cost once. Better: compile the reflected call to a delegate once — then invocation is near-native.

Example

```
var method = typeof(MyClass).GetMethod("DoWork");

// Compile once -- near-native invocation thereafter
var action = (Action<MyClass>)Delegate.CreateDelegate(
typeof(Action<MyClass>), method);
action(instance);
```

Q54 How do source generators compare to reflection?

Answer

Source generators run at compile time and emit C# code. The result is strongly typed, AOT-compatible, and has zero runtime reflection. System.Text.Json source-gen mode and Mapperly both use this. Trade-off: more complex build setup, but huge runtime gains.

Example

```
[JsonSerializable(typeof(Order))]
public partial class AppJsonContext : JsonSerializerContext { }

// Reflection-free at runtime:
var json = JsonSerializer.Serialize(order, AppJsonContext.Default.Order);
```

Ch 16 — Span, Memory, Pipelines

Q55 What is Span<T> and what problem does it solve?

Answer

Span<T> is a stack-only struct that references a contiguous memory block — array slice, stack memory, or unmanaged memory — without copying. Parsing, slicing, and in-place processing become zero-allocation operations.

Example

```
string text = "Hello, World";
ReadOnlySpan<char> hello = text.AsSpan(0, 5);
Console.WriteLine(hello.ToString()); // "Hello" -- no heap alloc
```

Q56 Why can't Span<T> be stored as a field on a class?

Answer

Span<T> is a ref struct — it must live on the stack. A class lives on the heap. Storing a Span inside a class would let it outlive the stack memory it points to — a use-after-free. Use Memory<T> instead: it lives on the heap and converts to Span on demand.

Example

```
class Buffer {
// Span<byte> _data; // compile error
Memory<byte> _data; // heap-safe
public Span<byte> AsSpan() => _data.Span;
}
```

Q57 What is ArrayPool and when should you use it?

Answer

ArrayPool<T>.Shared rents arrays from a pre-allocated pool. Return them when done and the memory is reused. Use it whenever you need a large temporary buffer — network reads, serialisation — to avoid LOH allocations.

Example

```
byte[] buf = ArrayPool<byte>.Shared.Rent(4096);
try {
int read = await stream.ReadAsync(buf.AsMemory());
Process(buf.AsSpan(0, read));
}
finally { ArrayPool<byte>.Shared.Return(buf); }
```

Ch 17 — Serialization

Q58 What is source-generator mode in System.Text.Json?

Answer

It generates all serialisation/deserialisation code at compile time — no runtime reflection. Use it for Native AOT (where reflection-based serialisation doesn't work) or high-throughput hot paths.

Example

```
[JsonSerializable(typeof(Order))]
[JsonSerializable(typeof(List<Order>))]
public partial class MyContext : JsonSerializerContext { }

var json = JsonSerializer.Serialize(order, MyContext.Default.Order);
```

Q59 Why was BinaryFormatter deprecated?

Answer

BinaryFormatter can execute arbitrary code during deserialisation — a well-known attack vector. It's also brittle for versioning as it couples serialised bytes to internal type layout. Use System.Text.Json for JSON, MessagePack for binary, or Protobuf for cross-language binary.

Example

```
// Throws in .NET 9+
// BinaryFormatter.Serialize(stream, obj);
```

```
var json = JsonSerializer.Serialize(obj);
var bytes = MessagePackSerializer.Serialize(obj);
```

Ch 18 — C# Version Features

Q60 What is the field keyword in C# 14?

Answer

field gives access to the compiler-generated backing field inside a property accessor without declaring the field yourself. Useful for semi-auto properties — e.g., trim on get, null-check on set.

Example

```
public string Name
{
get => field.Trim();
set => field = value ?? throw new ArgumentNullException();
}
```

Q61 What is CallerArgumentExpression and how is it useful?

Answer

CallerArgumentExpression captures the source-code text of an argument as a string at compile time. ArgumentNullException.ThrowIfNull uses this so the message automatically names which parameter was null.

Example

```
static void ThrowIfEmpty(
string value,
[CallerArgumentExpression(nameof(value))] string? argName = null)
{
if (string.IsNullOrEmpty(value))
throw new ArgumentException("Must not be empty", argName);
}
ThrowIfEmpty(name); // message: "Must not be empty (name)"
```

Ch 19 — .NET Runtime, CLR, JIT, AOT

Q62 What is tiered compilation in .NET?

Answer

On first call a method is JIT-compiled at Tier 0 — fast compile, fewer optimisations. After enough calls the runtime recompiles it at Tier 1 with full optimisations. Fast startup (cheap Tier 0) plus fast steady-state (optimised Tier 1) with no code changes needed.

Example

```
// On by default — no changes needed.
// Disable for benchmarking:
// DOTNET_TieredCompilation=0
```

Q63 What is Native AOT and what are the trade-offs?

Answer

Native AOT compiles the whole app to a standalone native binary — no JIT, no CLR bundled. Benefits: tiny startup, lower memory, single-file. Costs: no runtime code generation, limited dynamic features, longer build times, trimming can break libraries that use reflection.

Example

```
<!-- .csproj -->
<PublishAot>true</PublishAot>

// dotnet publish -r linux-x64 -c Release
// -> standalone native binary
```

Q64 What is PGO (profile-guided optimisation) and dynamic PGO?

Answer

PGO feeds runtime profiling data back into the JIT — which branches are taken, which types appear — to produce better-optimised code. Dynamic PGO does this automatically at runtime. On by default since .NET 8; gives meaningful throughput gains with zero code changes.

Example

```
// Dynamic PGO is on by default in .NET 8+.
// Check with: DOTNET_TieredPGO=1
// Observe with JitStats in BenchmarkDotNet
```

Ch 20 — Performance and Benchmarking

Q65 Why shouldn't you use Stopwatch for microbenchmarks?

Answer

Stopwatch doesn't account for JIT warm-up, CPU frequency scaling, GC pauses, or process noise. Your first measurement includes JIT compilation time. BenchmarkDotNet handles warm-up, statistical analysis, multiple iterations, GC pressure, and hardware counters.

Example

```
[MemoryDiagnoser]
public class MyBench {
[Benchmark] public string Concat() => "Hello" + " " + "World";
[Benchmark] public string Interpolate() => $"Hello World";
}
BenchmarkRunner.Run<MyBench>();
```

Q66 What does MemoryDiagnoser show and how do you read Gen columns?

Answer

Adds Allocated (bytes per op) and Gen0/Gen1/Gen2 (collections per 1,000 ops). Gen0=1 means one Gen 0 collection per 1,000 ops — usually fine. Any non-zero Gen2 means long-lived allocations in the expensive heap — investigate.

Example

```
// BenchmarkDotNet output:
// | Method | Mean | Gen0 | Allocated |
// |--------|--------|--------|-----------|
// | Alloc | 120 ns | 0.0234 | 96 B |
// ~23 Gen0 collections per 1,000 ops
```

Ch 21 — BCL and IO

Q67 What is the difference between Stream, MemoryStream, and FileStream?

Answer

Stream is the abstract base. MemoryStream is backed by an in-memory byte array — no I/O, great for testing. FileStream reads/writes a file on disk. Both implement Stream so code written against Stream works with either.

Example

```
async Task ProcessAsync(Stream input) {
byte[] buf = new byte[1024];
int n = await input.ReadAsync(buf);
// works with FileStream, MemoryStream, NetworkStream...
}
```

Q68 What is RandomAccess (introduced in .NET 6)?

Answer

RandomAccess provides static methods for reading/writing files at specific byte offsets without seeking. Multiple threads can safely read different parts of a large file concurrently — something FileStream can't do safely with its shared seek position.

Example

```
using var handle = File.OpenHandle("bigfile.bin");
byte[] buf = new byte[512];
await RandomAccess.ReadAsync(handle, buf, fileOffset: 4096);
// safe to call from multiple threads at different offsets
```

Ch 22 — NuGet

Q69 What is Central Package Management and Directory.Packages.props?

Answer

Central Package Management lets you declare every package version in one file at the repo root. Individual .csproj files reference packages without specifying a version — the central file supplies it, eliminating version drift across a large monorepo.

Example

```
<!-- Directory.Packages.props -->
<Project>
<ItemGroup>
<PackageVersion Include="Serilog" Version="3.1.1" />
<PackageVersion Include="Dapper" Version="2.1.28" />
</ItemGroup>
</Project>

<!-- In .csproj -- no version needed -->
<PackageReference Include="Serilog" />
```

Ch 23 — MSBuild and csproj

Q70 What is Directory.Build.props and what is it for?

Answer

MSBuild automatically imports this file from any ancestor directory. Put shared settings — Nullable, TreatWarningsAsErrors, TargetFramework — here once and they apply to every project under that folder. No more copying the same boilerplate into each .csproj.

Example

```
<!-- Directory.Build.props at repo root -->
<Project>
<PropertyGroup>
<Nullable>enable</Nullable>
```

```
<ImplicitUsings>enable</ImplicitUsings>
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>
</PropertyGroup>
</Project>
```

Q71 What is multi-targeting and when would you use it?

Answer

Multi-targeting builds one project against multiple framework versions in one pass, producing separate binaries for each. Use it in library projects that need to support both net8.0 and net9.0, or where you want to use newer APIs on newer runtimes with a fallback.

Example

```
<TargetFrameworks>net8.0;net9.0</TargetFrameworks>

#if NET9_0
// use net9-only API
#else
// fallback
#endif
```

Ch 24 — .NET CLI

Q72 What is the difference between dotnet build and dotnet publish?

Answer

dotnet build compiles code and resolves NuGet references — output is not self-contained. dotnet publish produces a ready-to-deploy folder with runtime files, static content, and any trim/single-file options you configure.

Example

```
dotnet build # compile, refs from local NuGet cache

dotnet publish -c Release -r linux-x64 --self-contained
# Full deployable folder with everything needed
```

Q73 What is global.json?

Answer

global.json pins the exact .NET SDK version for a directory tree. Without it, dotnet commands use the latest installed SDK — which silently breaks builds when teammates or CI have different versions.

Example

```
{
"sdk": {
"version": "9.0.100",
"rollForward": "latestPatch"
}
}
```

Q74 What is TaskCompletionSource<T> and when do you use it?

Answer

TaskCompletionSource<T> creates a Task you control manually — you decide when to set the result, throw an exception, or cancel it. Classic use: wrapping an old callback-based API into an awaitable Task.

Example

```
public Task<string> WaitForMessageAsync()
{
var tcs = new TaskCompletionSource<string>();
_socket.OnMessage += msg => tcs.SetResult(msg);
_socket.OnError += ex => tcs.SetException(ex);
return tcs.Task;
}
var msg = await WaitForMessageAsync();
```

Q75 What is the difference between Task.WhenAll and Task.WhenAny?

Answer

Task.WhenAll waits for ALL tasks to finish and collects all results. If any faults, the combined task faults. Task.WhenAny completes as soon as the FIRST task finishes — useful for timeout patterns or racing two sources.

Example

```
// WhenAll -- parallel fetch, correct syntax
var usersTask = GetUsersAsync();
var ordersTask = GetOrdersAsync();
await Task.WhenAll(usersTask, ordersTask);
var users = await usersTask; // already complete
var orders = await ordersTask;

// WhenAny -- timeout pattern
var fetchTask = FetchAsync();
var done = await Task.WhenAny(fetchTask, Task.Delay(3000));
if (done != fetchTask) throw new TimeoutException();
```

BUCKET 2 — ASP.NET CORE AND EF CORE

Ch 25 — Types of Web Projects

Q76 What are the main ASP.NET Core project types?

Answer

Empty Web -- bare canvas, no scaffolding. Web API (controller-based) -- MVC controllers returning JSON. Minimal API -- direct endpoint registration, no controllers. MVC with Views -- server-rendered HTML via Razor views. Razor Pages -- one page-model file per page. Blazor -- component-based UI (Server or WebAssembly). gRPC Service -- Protobuf-based typed RPC.

Example

```
// Minimal API -- entire app in a few lines
var app = WebApplication.Create(args);
app.MapGet("/ping", () => "pong");
app.Run();
```

Q77 When would you choose Minimal APIs over controller-based APIs?

Answer

Minimal APIs shine for microservices, small focused services, and serverless functions where you want the least ceremony possible. Controller-based APIs are better for larger teams wanting conventions, area support, a full filter pipeline, and easy discoverability across a big codebase.

Example

```
app.MapPost("/orders", async (CreateOrderDto dto, IOrderService svc) =>
Results.Created($"/orders/{dto.Id}", await svc.CreateAsync(dto)));
```

Ch 26 — Application Host

Q78 What is WebApplicationBuilder?

Answer

WebApplicationBuilder merges HostBuilder and WebHostBuilder into one fluent API. Register services on builder.Services, configure the host on builder.WebHost, call builder.Build() to get the WebApplication where you add middleware and map routes.

Example

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddDbContext<AppDb>(o => o.UseNpgsql(cs));
var app = builder.Build();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Q79 What is IHostedService and what is a typical use case?

Answer

IHostedService ties background work to the application lifetime. StartAsync fires at host start; StopAsync on graceful shutdown. Typical uses: message queue consumers, periodic cleanup jobs, warming up caches, seeding data.

Example

```
public class OutboxProcessor : BackgroundService {
protected override async Task ExecuteAsync(CancellationToken ct) {
while (!ct.IsCancellationRequested) {
await ProcessPendingMessages(ct);
await Task.Delay(TimeSpan.FromSeconds(5), ct);
}
}
}
```

Q80 How does IHost.StopAsync signal graceful shutdown?

Answer

StopAsync cancels the host lifetime CancellationToken. Every registered IHostedService.StopAsync is called with a deadline (default 5 seconds). After that the host terminates everything regardless.

Example

```
// Honour the token in your background loop:
while (!stoppingToken.IsCancellationRequested)
{
await DoWorkAsync(stoppingToken); // pass the token down
}
```

Ch 27 — Controllers

Q81 What is model binding in ASP.NET Core?

Answer

Model binding reads incoming data from route segments, query string, body, form fields, and headers, and maps them to action method parameters. The framework tries each source in a default order unless you pin one with an attribute.

Example

```
[HttpGet("{id}")]
public IActionResult Get(
int id, // from route
string filter, // from query string
[FromHeader] string token) // from header
{ ... }
```

Q82 Explain [FromBody], [FromQuery], [FromRoute], and [FromForm].

Answer

[FromBody] -- deserialises the request body (JSON/XML), read once. [FromQuery] -- reads a query-string parameter (?key=value). [FromRoute] -- reads a route template segment ({id}). [FromForm] -- reads a multipart/form-data or urlencoded form field.

Example

```
[HttpPost("upload")]
public IActionResult Upload(
[FromRoute] int tenantId,
[FromQuery] string tag,
[FromForm] IFormFile file)
{ ... }
```

Q83 What is the purpose of IActionResult?

Answer

IActionResult lets one action return different HTTP responses — Ok(), NotFound(), BadRequest(), Created() — depending on what happened. Returning a concrete type forces 200 OK always. Use ActionResult<T> when you want both type safety and the flexibility to return non-200 responses.

Example

```
public ActionResult<Order> Get(int id)
{
var order = _repo.Find(id);
if (order is null) return NotFound();
return Ok(order);
}
```

Q84 What is the lifecycle of a controller?

Answer

Controllers are Transient by default — created fresh per request with all dependencies injected, then discarded when the request ends. You can change this by registering controllers as services, but Transient is the safe default.

Example

```
public class OrdersController : ControllerBase {
private readonly IOrderService _svc;
public OrdersController(IOrderService svc) => _svc = svc;
// Fresh _svc instance every request
}
```

Q85 How do you handle exceptions globally for all controllers?

Answer

Three main options: 1. UseExceptionHandler middleware -- catches anything unhandled, redirects to error endpoint. 2. IExceptionHandler (.NET 8+) -- clean interface, chain multiple handlers, first match wins. 3. Exception filter -- runs inside the MVC pipeline, has access to action context.

Example

```
public class GlobalExceptionHandler : IExceptionHandler {
public async ValueTask<bool> TryHandleAsync(
HttpContext ctx, Exception ex, CancellationToken ct)
{
ctx.Response.StatusCode = 500;
await ctx.Response.WriteAsJsonAsync(new { error = ex.Message });
return true;
}
}
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
app.UseExceptionHandler();
```

Ch 28 — Minimal APIs

Q86 How does a Minimal API differ from a controller-based API?

Answer

Minimal APIs register endpoints directly in Program.cs with MapGet/MapPost etc. No controller class, no [ApiController], no action conventions. They use the same underlying pipeline but with far less ceremony.

Example

```
app.MapGet("/users/{id}", async (int id, IUserService svc) => {
var user = await svc.GetAsync(id);
return user is null ? Results.NotFound() : Results.Ok(user);
});
```

Q87 How do you secure Minimal API endpoints?

Answer

Add UseAuthentication() and UseAuthorization() to the pipeline, then chain RequireAuthorization() onto individual endpoints or groups.

Example

```
app.UseAuthentication();
app.UseAuthorization();

app.MapGet("/admin", () => "secret").RequireAuthorization("AdminPolicy");
app.MapGet("/public", () => "open");
```

Q88 How do you add API versioning to Minimal APIs?

Answer

Use the Asp.Versioning.Http package. Create an ApiVersionSet and attach endpoints to specific versions using WithApiVersionSet and MapToApiVersion.

Example

```
var vs = app.NewApiVersionSet()
.HasApiVersion(new ApiVersion(1))
.HasApiVersion(new ApiVersion(2))
.Build();

app.MapGet("/orders", GetV1).WithApiVersionSet(vs).MapToApiVersion(1);
app.MapGet("/orders", GetV2).WithApiVersionSet(vs).MapToApiVersion(2);
```

Q89 What filters are supported by Minimal APIs?

Answer

Minimal APIs support IEndpointFilter added with AddEndpointFilter. Filters form a pipeline around the endpoint handler — useful for logging, validation, and auth checks. Traditional MVC action/resource filters don't apply here.

Example

```
app.MapPost("/orders", CreateOrder)
.AddEndpointFilter(async (ctx, next) => {
// before handler
var result = await next(ctx);
// after handler
return result;
});
```

Q90 What are the trade-offs of Minimal APIs in large enterprise apps?

Answer

Pros: less boilerplate, faster startup, great for microservices and vertical-slice architecture. Cons: no convention-based routing, no area support, harder to discover endpoints across a big codebase, simpler filter pipeline than full MVC.

Example

```
// Keep large Minimal API apps organized:
public static class OrderEndpoints {
public static void Map(IEndpointRouteBuilder app) {
app.MapGet("/orders", GetAll);
app.MapPost("/orders", Create);
}
}
```

Ch 29 — Middlewares

Q91 What is the order of middleware execution and why does it matter?

Answer

Middleware runs in registration order for the request and in reverse order for the response. Order matters because each middleware decides whether to call next(). UseAuthentication must come before UseAuthorization — establish identity before checking permissions.

Example

```
app.UseExceptionHandler(); // first -- wraps everything
app.UseHttpsRedirection();
app.UseAuthentication(); // identity before permissions
app.UseAuthorization();
app.MapControllers();
```

Q92 What are all the ways to create middleware in ASP.NET Core?

Answer

1. Inline lambda: app.Use((ctx, next) => ...) 2. Convention-based class with InvokeAsync (singleton lifecycle) 3. Factory-based: implement IMiddleware, created per request via DI 4. Terminal: app.Run() -- no next call 5. Registration: app.UseMiddleware<T>()

Example

```
public class TimingMiddleware(RequestDelegate next) {
public async Task InvokeAsync(HttpContext ctx) {
var sw = Stopwatch.StartNew();
await next(ctx);
Console.WriteLine($"Took {sw.ElapsedMilliseconds}ms");
}
}
app.UseMiddleware<TimingMiddleware>();
```

Q93 What is the difference between convention-based and factory-based middleware?

Answer

Convention-based is instantiated once at startup — effectively a singleton. Don't inject scoped services via its constructor. Factory-based (IMiddleware) is resolved from DI on every request, so injecting scoped services in the constructor is safe.

Example

```
public class AuditMiddleware : IMiddleware {
// Scoped service is fine -- created per request
public AuditMiddleware(IAuditService audit) { ... }
public Task InvokeAsync(HttpContext ctx, RequestDelegate next) { ... }
}
builder.Services.AddScoped<AuditMiddleware>();
```

Q94 How do you handle exceptions globally using middleware?

Answer

Register UseExceptionHandler before everything else. It catches any unhandled exception, resets the response, and runs an error handler that writes the error response.

Example

```
app.UseExceptionHandler(errApp => {
errApp.Run(async ctx => {
var ex = ctx.Features.Get<IExceptionHandlerFeature>()?.Error;
```

```
ctx.Response.StatusCode = 500;
await ctx.Response.WriteAsJsonAsync(new { error = ex?.Message });
});
});
```

Ch 30 — Filters

Q95 What are the main filter types in ASP.NET Core?

Answer

Authorization filters -- run first; short-circuit if not allowed. Resource filters -- run after auth but before model binding; can return cached response early. Action filters -- run around the action method after model binding. Exception filters -- catch exceptions from action methods. Result filters -- run around IActionResult execution.

Example

```
public class LogActionFilter : IActionFilter {
public void OnActionExecuting(ActionExecutingContext ctx)
=> Console.WriteLine($"Executing {ctx.ActionDescriptor.DisplayName}");
public void OnActionExecuted(ActionExecutedContext ctx)
=> Console.WriteLine("Done");
}
```

Q96 How do you control filter execution order?

Answer

Implement IOrderedFilter and set the Order property. Lower numbers run earlier in the before-phase and later in the after-phase (outermost wrapper). You can also set order when registering global filters.

Example

```
public class FirstFilter : IActionFilter, IOrderedFilter {
public int Order => -1000; // runs before all default filters
}
```

Q97 What is the difference between resource filters and action filters?

Answer

Resource filters fire before model binding — return a cached response without the framework ever reading the body. Action filters fire after model binding — arguments are already populated, so you can inspect or mutate them.

Example

```
public class CacheFilter : IResourceFilter {
public void OnResourceExecuting(ResourceExecutingContext ctx) {
if (_cache.TryGet(ctx.HttpContext.Request.Path, out var cached))
ctx.Result = new ContentResult { Content = cached };
}
public void OnResourceExecuted(ResourceExecutedContext ctx) { }
}
```

Q98 What are filter scopes and how is precedence resolved?

Answer

Global filters apply to every action. Controller-level apply to all actions in that controller. Action-level apply to one action. When Order values tie, global runs outermost, then controller, then action.

Example

```
// Global
builder.Services.AddControllers(o => o.Filters.Add<LogFilter>());
// Controller-level
[ServiceFilter(typeof(ValidateTenantFilter))]
public class OrdersController : ControllerBase { }
// Action-level
[HttpPost, ValidateModel]
public IActionResult Create(OrderDto dto) { ... }
```

Ch 31 — REST Fundamentals

Q99 What is the Richardson Maturity Model?

Answer

Level 0 -- single URL, single verb (RPC over HTTP). Level 1 -- separate URLs per resource. Level 2 -- correct HTTP verbs and proper status codes. Level 3 -- HATEOAS: responses include links to next possible actions. Most real-world REST APIs target Level 2.

Example

```
// Level 2:
GET /orders -> 200 list
GET /orders/1 -> 200 single
POST /orders -> 201 created
PUT /orders/1 -> 200 updated
DELETE /orders/1 -> 204 no content
```

Q100 What are the six architectural constraints of REST?

Answer

1. Client-Server -- independent evolution of client and server. 2. Stateless -- each request contains all context the server needs; no server-side session. 3. Cacheable -- responses declare whether they can be cached. 4. Uniform Interface -- resource identified by URI, self-descriptive messages. 5. Layered System -- client can't tell if talking directly to server or proxy. 6. Code on Demand (optional) -- server can send executable code to client.

Example

```
// Stateless in practice:
// Bad: store user context in server-side session
// Good: send JWT in every request
```

Q101 What is the difference between PUT and PATCH?

Answer

PUT replaces the entire resource — any omitted field gets null or its default. PATCH applies a partial update — only the fields you send are changed.

Example

```
// PUT -- send the whole replacement
PUT /orders/1
{ "product": "Book", "quantity": 2, "status": "Paid" }

// PATCH -- only what changed
PATCH /orders/1
{ "status": "Shipped" }
```

Q102 What is idempotency and which HTTP methods are idempotent?

Answer

An operation is idempotent if calling it once or many times leaves the server in the same state. GET, PUT, DELETE, HEAD, OPTIONS are idempotent. POST is not — each call may create a new resource.

Example

```
DELETE /orders/1 // first: deletes. Second: 404.
// Server state is the same both times -> idempotent

POST /orders // each call creates a new order -> NOT idempotent
```

Q103 What is the difference between 401 and 403?

Answer

401 means no valid credentials — you aren't authenticated yet. 403 means credentials are valid but you don't have permission for this resource — authenticated but not authorised.

Example

```
// 401: missing or expired token
GET /admin/reports
Authorization: Bearer expired_token -> 401

// 403: valid token, wrong role
GET /admin/reports
Authorization: Bearer valid_user_token -> 403
```

Q104 What is HATEOAS?

Answer

HATEOAS (Hypermedia As The Engine Of Application State) means responses include links describing what actions the client can take next. The client discovers the API dynamically — no hard-coded URLs. This is REST Level 3.

Example

```
{
"id": 1, "status": "Pending",
"_links": {
"pay": { "href": "/orders/1/pay", "method": "POST" },
"cancel": { "href": "/orders/1/cancel", "method": "DELETE" }
}
}
```

Q105 What are common pagination strategies in REST APIs?

Answer

Offset pagination (?page=2&pageSize=20) is simple but slow for large offsets. Cursor/keyset pagination (?after=lastId&pageSize=20) uses an index seek — fast and stable for live data. Link header pagination includes prev/next URLs in response headers (GitHub style).

Example

```
GET /orders?after=550&pageSize=20
{
"data": [...],
"nextCursor": "570",
"hasMore": true
}
```

Q106 What is data shaping in REST APIs?

Answer

Data shaping lets the client request only needed fields via ?fields=id,name,email. Reduces payload, cuts bandwidth, and can be faster if also projected at the DB level so unused columns are never selected.

Example

```
GET /users?fields=id,email
[
{ "id": 1, "email": "alice@example.com" },
{ "id": 2, "email": "bob@example.com" }
]
```

Ch 32 — API Versioning

Q107 What are the four API versioning strategies?

Answer

URL (/api/v1/orders) -- most visible, easy to test in a browser. Query string (?api-version=1.0) -- clean URL, version is optional. Header (api-version: 1.0) -- URL stays clean, needs tool support to discover. Content negotiation (Accept: application/json;v=1.0) -- fully RESTful but complex.

Example

```
builder.Services.AddApiVersioning(o => {
o.ApiVersionReader = ApiVersionReader.Combine(
new UrlSegmentApiVersionReader(),
new HeaderApiVersionReader("api-version"));
});
```

Q108 How do you mark an API version as deprecated?

Answer

Call HasDeprecatedApiVersion() when building the ApiVersionSet. The framework adds a Deprecation response header and marks the version as deprecated in generated OpenAPI docs.

Example

```
var vs = app.NewApiVersionSet()
.HasApiVersion(new ApiVersion(2))
.HasDeprecatedApiVersion(new ApiVersion(1))
.Build();
```

Q109 How do you maintain backward compatibility across API versions?

Answer

Never remove or rename fields in an existing version. Only add new fields in new versions. Keep old endpoints live until all consumers have migrated. Use Sunset headers to give advance notice of retirement.

Example

```
// V1 -- frozen forever
{ "id": 1, "name": "Alice" }

// V2 -- additive only
{ "id": 1, "name": "Alice", "email": "alice@example.com" }
```

Ch 33 — Validation

Q110 What is FluentValidation and why use it over DataAnnotations?

Answer

FluentValidation lets you write validation rules in code rather than decorating models with attributes. Rules live in a separate validator class — proper separation of concerns. It supports conditional rules, async validation, and service injection, none of which DataAnnotations does cleanly.

Example

```
public class CreateOrderValidator : AbstractValidator<CreateOrderDto> {
public CreateOrderValidator() {
RuleFor(x => x.ProductId).GreaterThan(0);
RuleFor(x => x.Quantity).InclusiveBetween(1, 100);
RuleFor(x => x.Email).NotEmpty().EmailAddress();
}
}
```

Q111 How does FluentValidation handle nested objects and collections?

Answer

Use SetValidator to delegate to a child validator for a nested object. Use RuleForEach to run a validator against every item in a collection.

Example

```
RuleFor(x => x.Address).SetValidator(new AddressValidator());
RuleForEach(x => x.Items).SetValidator(new OrderItemValidator());
```

Q112 How do you perform async validation in FluentValidation?

Answer

Use MustAsync in the rule builder. Always call ValidateAsync on the validator — the synchronous Validate() simply skips async rules.

Example

```
RuleFor(x => x.Email)
.MustAsync(async (email, ct) =>
!await _db.Users.AnyAsync(u => u.Email == email, ct))
.WithMessage("Email already taken");

var result = await _validator.ValidateAsync(dto);
```

Q113 How do you inject services into a FluentValidation validator?

Answer

Register the validator in DI and accept services through its constructor. Use AddFluentValidationAutoValidation() so ASP.NET Core runs the validator automatically on model binding.

Example

```
public class UniqueEmailValidator : AbstractValidator<RegisterDto> {
public UniqueEmailValidator(AppDbContext db) {
RuleFor(x => x.Email)
.MustAsync(async (e, ct) =>
!await db.Users.AnyAsync(u => u.Email == e, ct));
}
}
```

Q114 How do you return FluentValidation errors as Problem Details?

Answer

Register AddFluentValidationAutoValidation(). When validation fails, ASP.NET Core returns a ValidationProblemDetails JSON response (RFC 9457 format) with field errors grouped by property name.

Example

```
builder.Services.AddFluentValidationAutoValidation();
builder.Services.AddScoped<IValidator<CreateOrderDto>, CreateOrderValidator>();
// On failure:
// { "title": "One or more validation errors occurred.",
// "errors": { "Email": ["Email already taken"] } }
```

Ch 34 — Mapping

Q115 What is AutoMapper and what are common pitfalls?

Answer

AutoMapper maps properties by convention at runtime using reflection. Common problems: silent mismatches when property names differ, hard-to-trace global configuration, N+1 queries when mapping inside LINQ instead of projecting at the DB, and reflection overhead on hot paths.

Example

```
var config = new MapperConfiguration(cfg =>
cfg.CreateMap<Order, OrderDto>());
var mapper = config.CreateMapper();
OrderDto dto = mapper.Map<OrderDto>(order);
```

Q116 What is Mapperly?

Answer

Mapperly is a source-generator mapper — it generates the full mapping code at compile time from a partial class you annotate. Zero runtime reflection, AOT-safe, and the generated code is human-readable.

Example

```
[Mapper]
public partial class OrderMapper {
public partial OrderDto ToDto(Order order);
}
// Compile-time generated:
// public partial OrderDto ToDto(Order o)
// => new OrderDto { Id = o.Id, ... };
```

Q117 Mapperly vs AutoMapper -- what are the trade-offs?

Answer

Mapperly: compile-time errors if a mapping is wrong, no reflection, AOT-safe, faster. AutoMapper: more flexible conventions, easier for large legacy codebases, but runtime-only errors and reflection-based.

Example

```
// Mapperly: compile error if a property is unmapped
// AutoMapper: silently ignores unmapped properties unless you call
// AssertConfigurationIsValid()
```

Q118 When is manual mapping better than a mapping library?

Answer

When the mapping has non-trivial business logic — conditional fields, computed values, lookups. Manual mapping via extension methods is explicit, zero-dependency, and instantly debuggable.

Example

```
public static OrderDto ToDto(this Order o) => new() {
Id = o.Id,
Total = o.Items.Sum(i => i.Price * i.Qty),
StatusLabel = o.Status == Status.Paid ? "Paid" : "Pending"
```

```
};
```

Ch 35 — Configuration and Options Pattern

Q119 What is the options pattern and why prefer it over IConfiguration?

Answer

The options pattern binds a config section to a typed POCO. You inject IOptions<T>, IOptionsSnapshot<T>, or IOptionsMonitor<T> instead of raw IConfiguration strings. Benefits: strongly typed, validated at startup, reloadable, and easy to unit test.

Example

```
public class SmtpOptions {
public string Host { get; set; } = "";
public int Port { get; set; } = 587;
}
builder.Services.Configure<SmtpOptions>(
builder.Configuration.GetSection("Smtp"));

public class MailService(IOptions<SmtpOptions> opts) {
private readonly SmtpOptions _smtp = opts.Value;
}
```

Q120 What is the difference between IOptions, IOptionsSnapshot, and IOptionsMonitor?

Answer

IOptions<T> -- singleton; values fixed at startup. Simplest option. IOptionsSnapshot<T> -- scoped; re-reads config once per HTTP request. IOptionsMonitor<T> -- singleton with OnChange callback; notified immediately when config changes. Use in background services.

Example

```
public class Worker(IOptionsMonitor<FeatureFlags> monitor) {
public Worker() =>
monitor.OnChange(flags => Console.WriteLine("Flags reloaded"));
}
```

Q121 How do you validate options at startup with DataAnnotations?

Answer

Call ValidateDataAnnotations() and ValidateOnStart(). The framework validates the bound POCO at startup and throws immediately if any annotations are violated — fail fast.

Example

```
public class SmtpOptions {
[Required] public string Host { get; set; } = "";
[Range(1, 65535)] public int Port { get; set; }
}
builder.Services.AddOptions<SmtpOptions>()
.BindConfiguration("Smtp")
.ValidateDataAnnotations()
.ValidateOnStart();
```

Q122 How do you keep secrets out of appsettings?

Answer

Dev: User Secrets (dotnet user-secrets) -- stored outside the repo. Production: environment variables, Azure Key Vault, AWS Secrets Manager, or HashiCorp Vault. The config system layers multiple providers, so secrets override

appsettings.

Example

```
// Azure Key Vault
builder.Configuration.AddAzureKeyVault(
new Uri(builder.Configuration["KeyVault:Uri"]!),
new DefaultAzureCredential());
```

Ch 36 — Error Handling

Q123 What is UseExceptionHandler middleware?

Answer

UseExceptionHandler wraps the rest of the pipeline in a try/catch. When an unhandled exception escapes it resets the response and runs an error handler. Register it first so it catches errors from all other middleware.

Example

```
app.UseExceptionHandler("/error");
app.Map("/error", (HttpContext ctx) => {
var ex = ctx.Features.Get<IExceptionHandlerFeature>()?.Error;
return Results.Problem(detail: ex?.Message, statusCode: 500);
});
```

Q124 What is IExceptionHandler in .NET 8?

Answer

A clean interface — implement TryHandleAsync, return true if you handled it, false to pass to the next handler. Chain multiple handlers for different exception types. Integrates nicely with Problem Details.

Example

```
public class NotFoundHandler : IExceptionHandler {
public async ValueTask<bool> TryHandleAsync(
HttpContext ctx, Exception ex, CancellationToken ct)
{
if (ex is not NotFoundException) return false;
ctx.Response.StatusCode = 404;
await ctx.Response.WriteAsJsonAsync(
new ProblemDetails { Title = "Not found", Status = 404 });
return true;
}
}
```

Q125 How do you return Problem Details (RFC 9457) in ASP.NET Core?

Answer

Call AddProblemDetails() and UseExceptionHandler(). ASP.NET Core returns ProblemDetails JSON for 4xx/5xx responses automatically. Return Results.Problem() from Minimal APIs or Problem() from controllers for manual control.

Example

```
builder.Services.AddProblemDetails();

// Manual:
return Results.Problem(
title: "Validation failed",
statusCode: 422,
detail: "Quantity must be positive");
```

Q126 How do you map custom exceptions to HTTP status codes globally?

Answer

Use IExceptionHandler with a switch expression to map known domain exception types to status codes.

Example

```
public class DomainExceptionHandler : IExceptionHandler {
public async ValueTask<bool> TryHandleAsync(
HttpContext ctx, Exception ex, CancellationToken ct)
{
var status = ex switch {
NotFoundException => 404,
ValidationException => 422,
ConflictException => 409,
_ => 0
};
if (status == 0) return false;
ctx.Response.StatusCode = status;
await ctx.Response.WriteAsJsonAsync(
new ProblemDetails { Status = status, Detail = ex.Message });
return true;
}
}
```

Ch 37 — Logging

Q127 What is structured logging and why does it matter?

Answer

Structured logging stores log entries as key-value pairs rather than plain strings. When you pass a named placeholder like {OrderId}, the logging framework saves the value as a separate field. Log aggregators like Seq or Elasticsearch can then filter on orderId == 1234 instead of doing regex on a text blob.

Example

```
// Structured -- OrderId stored as searchable field
_logger.LogInformation("Order {OrderId} shipped to {City}",
order.Id, order.City);

// Unstructured -- just a string, can't filter without regex
_logger.LogInformation($"Order {order.Id} shipped to {order.City}");
```

Q128 What are logging scopes and when do you use them?

Answer

A logging scope attaches extra key-value pairs to all log entries made within a using block. It is the right way to add RequestId, UserId, or TenantId to every line inside an operation without passing those values to every method.

Example

```
using (_logger.BeginScope(new Dictionary<string, object> {
["RequestId"] = ctx.TraceIdentifier,
["UserId"] = userId
}))
{
_logger.LogInformation("Processing order"); // both fields appear
await ProcessAsync();
}
```

Q129 How do you enable SQL logging in EF Core?

Answer

Set the EF Core command log level to Information in your logging config. For development also call EnableSensitiveDataLogging() to include parameter values — never use that in production.

Example

```
{
"Logging": { "LogLevel": {
"Microsoft.EntityFrameworkCore.Database.Command": "Information"
}}
}
// Dev only:
optionsBuilder.EnableSensitiveDataLogging().LogTo(Console.WriteLine);
```

Q130 How do you set up centralised log aggregation in production?

Answer

Typical stack: Serilog as the library, structured JSON sinks (Seq locally, Elasticsearch/Loki/CloudWatch in cloud), correlation IDs on every entry, and alerts in Grafana or Kibana on elevated error rates.

Example

```
Log.Logger = new LoggerConfiguration()
.Enrich.FromLogContext()
.Enrich.WithCorrelationId()
.WriteTo.Console(new JsonFormatter())
.WriteTo.Seq("http://seq:5341")
.CreateLogger();
builder.Host.UseSerilog();
```

Q131 How do you configure log file rotation in Serilog?

Answer

Use WriteTo.File with rollingInterval and fileSizeLimitBytes. retainedFileCountLimit controls how many old files to keep before Serilog deletes the oldest.

Example

```
Log.Logger = new LoggerConfiguration()
.WriteTo.File("logs/app.log",
rollingInterval: RollingInterval.Day,
fileSizeLimitBytes: 50_000_000,
retainedFileCountLimit: 14,
rollOnFileSizeLimit: true)
.CreateLogger();
```

Ch 38 — Health Checks

Q132 What is the difference between a liveness probe and a readiness probe?

Answer

Liveness: is the process alive? If not, Kubernetes restarts the container. Readiness: is the app ready to serve traffic? If not, Kubernetes removes the pod from load balancer rotation but does not restart it. A starting app should fail readiness but pass liveness.

Example

```
// Liveness -- just 'am I alive?'
app.MapHealthChecks("/health/live", new() { Predicate = _ => false });
// Readiness -- run all registered checks
```

```
app.MapHealthChecks("/health/ready", new() { });
```

Q133 How do you implement a custom health check?

Answer

Implement IHealthCheck and return Healthy, Degraded, or Unhealthy. Register with AddHealthChecks().AddCheck<T>().

Example

```
public class RedisHealthCheck : IHealthCheck {
private readonly IDatabase _db;
public RedisHealthCheck(IConnectionMultiplexer mux)
=> _db = mux.GetDatabase();
public async Task<HealthCheckResult> CheckHealthAsync(
HealthCheckContext ctx, CancellationToken ct)
{
var ok = await _db.PingAsync() < TimeSpan.FromSeconds(1);
return ok ? HealthCheckResult.Healthy()
: HealthCheckResult.Unhealthy("Redis slow");
}
}
builder.Services.AddHealthChecks().AddCheck<RedisHealthCheck>("redis");
```

Ch 39 — Dependency Injection

Q134 What are the three DI lifetimes in ASP.NET Core?

Answer

Transient -- new instance every time requested. Use for stateless services. Scoped -- one instance per HTTP request (or per DI scope). DbContext is the classic example. Singleton -- one instance for the entire app lifetime. Use for caches, configuration, HttpClientFactory.

Example

```
builder.Services.AddTransient<IEmailSender, SmtpEmailSender>();
builder.Services.AddScoped<IOrderRepository, EfOrderRepository>();
builder.Services.AddSingleton<ICache, MemoryCache>();
```

Q135 What is a captive dependency?

Answer

A captive dependency happens when a singleton captures a scoped service in its constructor. The scoped service is supposed to be created and discarded per request, but being held by a singleton means it lives forever. ASP.NET Core throws InvalidOperationException at startup when it detects this.

Example

```
// Bad -- will throw at startup
builder.Services.AddSingleton<ReportGenerator>();
// ReportGenerator depends on scoped IOrderRepository

// Fix -- use IServiceScopeFactory to create a scope when needed
public class ReportGenerator(IServiceScopeFactory factory) { ... }
```

Q136 How do you use a scoped service inside a background task?

Answer

Inject IServiceScopeFactory into the singleton or background service. Create a new scope around each unit of work and resolve the scoped service from that scope.

Example

```
public class OutboxWorker(IServiceScopeFactory factory) : BackgroundService {
protected override async Task ExecuteAsync(CancellationToken ct) {
while (!ct.IsCancellationRequested) {
using var scope = factory.CreateScope();
var repo = scope.ServiceProvider
.GetRequiredService<IOutboxRepo>();
await repo.ProcessAsync(ct);
await Task.Delay(5000, ct);
}
}
}
```

Q137 How do you fix circular dependencies in DI?

Answer

Circular dependencies usually signal a design problem. Cleanest fix: extract a third class both can depend on, or use events/mediator to decouple them. Quick fix: Lazy<T> breaks the cycle by deferring resolution until first use.

Example

```
public class ServiceA {
private readonly Lazy<IServiceB> _b;
public ServiceA(Lazy<IServiceB> b) => _b = b;
public void DoWork() => _b.Value.Help();
}
```

Q138 How do you conditionally resolve different implementations?

Answer

Register all implementations, inject IEnumerable<T>, and pick by a key property at runtime. Alternatively, use a factory delegate.

Example

```
builder.Services.AddScoped<IPaymentGateway, StripeGateway>();
builder.Services.AddScoped<IPaymentGateway, PayPalGateway>();

public class Checkout(IEnumerable<IPaymentGateway> gateways) {
public Task Pay(string provider) =>
gateways.First(g => g.Name == provider).ChargeAsync();
}
```

Ch 43 — Entity Framework Core

Q139 What is the difference between DbContext and DbSet<T>?

Answer

DbContext is the unit of work: tracks changes, manages identity map, coordinates SaveChanges. DbSet<T> represents one table. You write LINQ against DbSet; DbContext translates and executes them.

Example

```
public class AppDbContext : DbContext {
public DbSet<Order> Orders { get; set; }
public DbSet<Product> Products { get; set; }
}
var paid = await db.Orders
.Where(o => o.Total > 100).ToListAsync();
```

Q140 Explain eager, lazy, and explicit loading.

Answer

Eager (Include) -- loads related entities in the same query via JOIN. No extra round-trips. Lazy -- related entities load automatically when you access the navigation property (requires proxies). Risk: N+1 queries. Explicit -- you call LoadAsync manually when you decide you need the related data.

Example

```
// Eager
var orders = await db.Orders.Include(o => o.Items).ToListAsync();

// Explicit -- you choose when to load
var order = await db.Orders.FindAsync(id);
await db.Entry(order).Collection(o => o.Items).LoadAsync();
```

Q141 When and why should you use AsNoTracking()?

Answer

Use it for read-only queries — GET endpoints, reports — where you won't update the returned entities. EF Core skips adding them to the change tracker, reducing both memory and CPU overhead.

Example

```
var orders = await db.Orders
.AsNoTracking()
.Where(o => o.Status == "Paid")
.ToListAsync();
```

Q142 What is AsSplitQuery and what does it solve?

Answer

When you Include multiple collection navigations EF Core generates a cartesian-product JOIN. The result set explodes: 1 order x 100 items x 50 tags = 5,000 rows. AsSplitQuery fires a separate SQL query per collection, avoiding the explosion at the cost of extra round-trips.

Example

```
var orders = await db.Orders
.Include(o => o.Items)
.Include(o => o.Tags)
.AsSplitQuery()
.ToListAsync();
```

Q143 What is DbContext pooling?

Answer

AddDbContextPool reuses DbContext instances across requests instead of creating and GC-ing one per request. Between uses the context is reset — change tracker cleared. This cuts GC pressure significantly on high-throughput services.

Example

```
builder.Services.AddDbContextPool<AppDbContext>(
o => o.UseNpgsql(connectionString),
poolSize: 128);
```

Q144 How do you implement optimistic concurrency with EF Core?

Answer

Add a RowVersion byte[] property and mark it [Timestamp]. EF Core includes the token in UPDATE WHERE clauses. If another process changed the row first, the UPDATE matches 0 rows and EF throws DbUpdateConcurrencyException — catch it and retry.

Example

```
public class Order {
public int Id { get; set; }
[Timestamp] public byte[] RowVersion { get; set; }
}
try { await db.SaveChangesAsync(); }
catch (DbUpdateConcurrencyException) { /* reload and retry */ }
```

Q145 How do you do bulk update/delete without loading entities?

Answer

EF Core 7 added ExecuteUpdateAsync and ExecuteDeleteAsync. These emit a single UPDATE or DELETE SQL statement directly — no entities loaded, no change tracking, no SaveChanges needed.

Example

```
await db.Orders
.Where(o => o.Status == "Cancelled"
&& o.CreatedAt < DateTime.UtcNow.AddYears(-1))
.ExecuteDeleteAsync();

await db.Products
.Where(p => p.Category == "Electronics")
.ExecuteUpdateAsync(s => s.SetProperty(p => p.Discount, 0.1m));
```

BUCKET 3 — CACHING, SCHEDULING, MESSAGING & SECURITY

Ch 40 — Authentication and Authorization

Q146 What is the difference between authentication and authorization?

Answer

Authentication answers WHO ARE YOU -- verifying identity (JWT, cookie, API key). Authorization answers WHAT CAN YOU DO -- checking what the authenticated identity is allowed to do. The two middleware must be registered in that order: authenticate first, then authorise.

Example

```
app.UseAuthentication(); // who are you?
app.UseAuthorization(); // what can you do?

[Authorize(Roles = "Admin")]
public IActionResult DeleteUser(int id) { ... }
```

Q147 What is the difference between cookie auth and JWT bearer?

Answer

Cookie auth stores the session token in an HTTP-only cookie -- the browser attaches it automatically. Ideal for server-rendered web apps. JWT bearer puts the token in the Authorization header -- the client must manage it. Ideal for SPAs, mobile, and service-to-service calls.

Example

```
// Cookie
builder.Services.AddAuthentication(
CookieAuthenticationDefaults.AuthenticationScheme).AddCookie();

// JWT Bearer
builder.Services.AddAuthentication(
JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(o => o.Authority = "https://identity.example.com");
```

Q148 What are refresh tokens and how does the JWT refresh flow work?

Answer

Access tokens are short-lived (minutes). Refresh tokens are long-lived (days/weeks). Flow: 1. Login -> server returns access token + refresh token. 2. Access token expires -> client POSTs refresh token to /auth/refresh. 3. Server validates and returns new tokens. 4. On logout -> server revokes the refresh token.

Example

```
POST /auth/login -> { accessToken (5min), refreshToken (7d) }
POST /auth/refresh -> body: { refreshToken }
-> { new accessToken, new refreshToken }
```

Q149 How do you implement policy-based authorization?

Answer

Define a named policy with AddAuthorization, specify what claims or roles are required, apply the policy name with [Authorize(Policy='name')].

Example

```
builder.Services.AddAuthorization(o =>
o.AddPolicy("CanApproveOrders",
```

```
p => p.RequireClaim("permission", "orders:approve")));

[Authorize(Policy = "CanApproveOrders")]
public IActionResult Approve(int id) { ... }
```

Q150 What is an authorization Requirement?

Answer

A Requirement is a plain marker class carrying data the authorization decision needs. A separate AuthorizationHandler reads it and calls ctx.Succeed or ctx.Fail.

Example

```
public class MinimumAgeRequirement : IAuthorizationRequirement {
public int MinAge { get; }
public MinimumAgeRequirement(int min) => MinAge = min;
}
public class AgeHandler : AuthorizationHandler<MinimumAgeRequirement> {
protected override Task HandleRequirementAsync(
AuthorizationHandlerContext ctx, MinimumAgeRequirement req)
{
var dob = ctx.User.FindFirst("dob");
if (dob != null && CalculateAge(dob.Value) >= req.MinAge)
ctx.Succeed(req);
return Task.CompletedTask;
}
}
```

Q151 How do you create a custom authorization handler?

Answer

Inherit AuthorizationHandler<TRequirement>, override HandleRequirementAsync, call ctx.Succeed when satisfied or ctx.Fail to hard-fail. Register the handler as a service.

Example

```
builder.Services.AddScoped<IAuthorizationHandler, AgeHandler>();
builder.Services.AddAuthorization(o =>
o.AddPolicy("Over18",
p => p.AddRequirements(new MinimumAgeRequirement(18))));
```

Q152 When is claims transformation necessary and how do you implement it?

Answer

JWT tokens carry static claims baked in at login. If the user's role changes after login the token doesn't reflect it until expiry. IClaimsTransformation lets you augment claims on every request from the database.

Example

```
public class PermissionsTransformer : IClaimsTransformation {
public async Task<ClaimsPrincipal> TransformAsync(ClaimsPrincipal p) {
var userId = p.FindFirst("sub")?.Value;
var perms = await _db.GetPermissionsAsync(userId);
var id = new ClaimsIdentity();
foreach (var perm in perms)
id.AddClaim(new Claim("permission", perm));
p.AddIdentity(id);
return p;
}
}
```

Ch 41 — ASP.NET Core Identity

Q153 How do you configure password requirements in Identity?

Answer

Set the IdentityOptions.Password properties in the AddIdentity lambda.

Example

```
builder.Services.AddIdentity<AppUser, IdentityRole>(o => {
o.Password.RequiredLength = 12;
o.Password.RequireDigit = true;
o.Password.RequireUppercase = true;
o.Password.RequireNonAlphanumeric = true;
});
```

Q154 How do you implement two-factor authentication with Identity?

Answer

Enable TwoFactorEnabled on the user, configure a token provider, use GenerateTwoFactorTokenAsync to create a code, and TwoFactorSignInAsync to validate it.

Example

```
var token = await _userManager
.GenerateTwoFactorTokenAsync(user, "Email");
await _emailSender.SendAsync(user.Email, "Your code", token);

var result = await _signInManager.TwoFactorSignInAsync(
"Email", code, isPersistent: false, rememberClient: false);
```

Q155 How do you lock out users after failed login attempts?

Answer

Enable lockout in IdentityOptions. Every failed PasswordSignInAsync call increments the failure counter. After MaxFailedAccessAttempts the account is locked for DefaultLockoutTimeSpan.

Example

```
o.Lockout.MaxFailedAccessAttempts = 5;
o.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
o.Lockout.AllowedForNewUsers = true;
// PasswordSignInAsync handles lockout automatically
```

Q156 How do you integrate Identity with JWT authentication?

Answer

After a successful Identity login, generate a JWT containing the user's claims with JwtSecurityTokenHandler. Return it to the client. Configure AddJwtBearer on the API to validate incoming tokens.

Example

```
var claims = new[] {
new Claim(JwtRegisteredClaimNames.Sub, user.Id),
new Claim(ClaimTypes.Email, user.Email)
};
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_jwtKey));
var token = new JwtSecurityToken(
issuer: "myapp", audience: "myapp", claims: claims,
expires: DateTime.UtcNow.AddMinutes(15),
signingCredentials: new SigningCredentials(
key, SecurityAlgorithms.HmacSha256));
```

Q157 How do you create a custom user store for a NoSQL database?

Answer

Implement IUserStore<T> plus whichever optional interfaces you need (IUserPasswordStore, IUserRoleStore). Register with AddIdentityCore().AddUserStore<T>(). UserManager delegates all persistence to your store.

Example

```
public class MongoUserStore :
IUserStore<AppUser>, IUserPasswordStore<AppUser>
{
private readonly IMongoCollection<AppUser> _col;
public MongoUserStore(IMongoDatabase db)
=> _col = db.GetCollection<AppUser>("users");
public Task<AppUser> FindByIdAsync(string id, CancellationToken ct)
=> _col.Find(u => u.Id == id).FirstOrDefaultAsync(ct);
// implement remaining interface members...
}
```

Ch 42 — ASP.NET Core Security

Q158 What is CORS and what is the risk of a permissive policy?

Answer

CORS controls which origins browsers allow to call your API. Allowing * combined with cookie auth is dangerous: any website can make credentialed requests on behalf of your users. Always restrict origins, methods, and headers to what you need.

Example

```
builder.Services.AddCors(o => o.AddPolicy("FrontEnd", p =>
p.WithOrigins("https://app.example.com")
.WithMethods("GET", "POST")
.AllowCredentials()));
app.UseCors("FrontEnd");
```

Q159 How do preflight OPTIONS requests work?

Answer

Before a cross-origin request with custom headers or non-simple verbs, the browser sends an OPTIONS preflight. The server must reply with Access-Control-Allow-* headers. If the preflight fails the browser blocks the real request. ASP.NET Core handles this automatically when CORS middleware is configured.

Example

```
OPTIONS /api/orders
Origin: https://app.example.com
Access-Control-Request-Method: POST
// Server replies:
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: POST
```

Q160 How do you validate a JWT's signature and claims?

Answer

Configure TokenValidationParameters in AddJwtBearer. The middleware validates signature, issuer, audience, and expiry on every incoming request automatically.

Example

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
.AddJwtBearer(o => {
```

```
o.TokenValidationParameters = new() {
ValidateIssuer = true, ValidIssuer = "myapp",
ValidateAudience = true, ValidAudience = "myapp",
ValidateLifetime = true,
ValidateIssuerSigningKey = true,
IssuerSigningKey = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(secret))
};
});
```

Q161 What are the security risks of refresh tokens?

Answer

If stolen, the attacker gets long-lived access. Mitigations: store only a hashed copy in the database, rotate on every use, bind to device fingerprint, set expiry, and revoke all tokens for a user on logout or password change.

Example

```
// Refresh token rotation:
// 1. Client sends old refresh token
// 2. Server validates, issues new access + refresh token
// 3. Server marks old token as revoked
// 4. If old token replayed -> revoke entire chain (replay attack)
```

Q162 How do you revoke refresh tokens on logout?

Answer

Store refresh tokens in the database with an IsRevoked flag. On logout or password change, mark all of that user's tokens as revoked. JWT access tokens can't be individually invalidated before expiry — the server-side store is the only way.

Example

```
await _db.RefreshTokens
.Where(t => t.UserId == userId)
.ExecuteUpdateAsync(s => s.SetProperty(t => t.IsRevoked, true));

if (token.IsRevoked) return Unauthorized();
```

Q163 What is OAuth 2.0?

Answer

OAuth 2.0 is a delegation framework. A user grants a client app limited access to their resources on a server, mediated by an Authorization Server, without handing the client their credentials. The client gets a scoped access token.

Example

```
// Authorization Code flow:
// 1. Redirect to /authorize?client_id=...&scope=orders:read
// 2. User authenticates and consents
// 3. Auth server redirects back with authorization code
// 4. App exchanges code for access token at /token
// 5. App calls API: Authorization: Bearer <token>
```

Q164 What is OpenID Connect and how does it differ from OAuth 2.0?

Answer

OAuth 2.0 is about authorization (access tokens). OpenID Connect adds authentication on top -- it returns an ID token (JWT) saying WHO the user is (name, email, subject). Use OAuth 2.0 for API access delegation. Use OIDC

for user login and SSO.

Example

```
{
"access_token": "...",
"id_token": "<JWT: sub, email, name>",
"token_type": "Bearer"
}
// access_token -> what you can do
// id_token -> who you are
```

Ch 44 — Caching

Q165 What is IDistributedCache and how does it differ from IMemoryCache?

Answer

IMemoryCache is in-process -- fast but private to one instance and lost on restart. IDistributedCache is backed by Redis, SQL Server, etc. -- shared across all instances and survives restarts. Use memory cache for single-instance; distributed cache for multi-instance deployments.

Example

```
// In-memory only
_memCache.Set("key", value, TimeSpan.FromMinutes(5));

// Shared across pods
await _distCache.SetStringAsync("key", json, new() {
AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5)
});
```

Q166 How do you configure Redis cache expiration and eviction?

Answer

Set TTL in DistributedCacheEntryOptions when writing. Configure Redis maxmemory-policy at the server level (allkeys-lru is a sensible default) for what gets evicted when memory fills up.

Example

```
await _cache.SetStringAsync("product:1", json,
new DistributedCacheEntryOptions {
AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(1),
SlidingExpiration = TimeSpan.FromMinutes(20)
});
```

Q167 What is OutputCache and how do you use it?

Answer

OutputCache stores the complete HTTP response for an endpoint and serves it directly to subsequent matching requests -- the action never runs again until the entry expires. Register AddOutputCache and UseOutputCache, annotate endpoints with CacheOutput.

Example

```
builder.Services.AddOutputCache();
app.UseOutputCache();

app.MapGet("/products", GetProducts)
.CacheOutput(p => p.Expire(TimeSpan.FromMinutes(10)));
```

Q168 How do you vary OutputCache by user or query parameter?

Answer

Use VaryByQuery, VaryByHeader, or VaryByValue in the cache policy. This creates a separate cache entry per unique combination of those values.

Example

```
builder.Services.AddOutputCache(o =>
o.AddPolicy("ByUser", p =>
p.VaryByValue(ctx =>
ctx.User.FindFirst("sub")?.Value ?? "anon")
.Expire(TimeSpan.FromSeconds(30))));
```

Q169 What is HybridCache in .NET 9?

Answer

HybridCache is a two-level cache: L1 is in-process memory for ultra-fast reads; L2 is a distributed cache (Redis). On a hit it serves from L1. On a miss it checks L2, populates L1, and returns. Write-through keeps both in sync.

Example

```
builder.Services.AddHybridCache();

var value = await _hybridCache.GetOrCreateAsync(
"product:1",
async ct => await _db.Products.FindAsync(1, ct),
new() { Expiration = TimeSpan.FromMinutes(5) });
```

Q170 How do you wire HybridCache with Redis as the L2 backend?

Answer

Register AddStackExchangeRedisCache for L2 and AddHybridCache for the orchestration. HybridCache automatically detects and uses the registered IDistributedCache (Redis) as its L2.

Example

```
builder.Services.AddStackExchangeRedisCache(
o => o.Configuration = "redis:6379");
builder.Services.AddHybridCache();
// in-memory = L1, Redis = L2 automatically
```

Q171 What is FusionCache and what does it add over HybridCache?

Answer

FusionCache adds two extras HybridCache currently lacks: 1. Cache stampede protection -- only one request fetches from source when cold; others wait. 2. Soft expiry / stale-while-revalidate -- serve a slightly stale value while refreshing in the background. It also adds a circuit breaker for the distributed cache layer.

Example

```
builder.Services.AddFusionCache()
.WithDefaultEntryOptions(o => o
.SetDuration(TimeSpan.FromMinutes(5))
.SetFailSafe(true)); // serve stale if downstream is down

var product = await _fusionCache.GetOrSetAsync(
"product:1",
async ct => await _db.Products.FindAsync(1, ct));
```

Ch 49 — Task Scheduling

Q172 What is a BackgroundService in ASP.NET Core?

Answer

BackgroundService is an abstract base class implementing IHostedService. Override ExecuteAsync and write your work loop there. The host starts it on app start and cancels its CancellationToken on graceful shutdown.

Example

```
public class CleanupService : BackgroundService {
protected override async Task ExecuteAsync(CancellationToken ct) {
while (!ct.IsCancellationRequested) {
await DeleteOldRecordsAsync(ct);
await Task.Delay(TimeSpan.FromHours(1), ct);
}
}
}
```

Q173 What is the difference between BackgroundService and IHostedService?

Answer

IHostedService is the low-level interface with StartAsync and StopAsync -- you manage everything. BackgroundService is a convenience wrapper: it runs your ExecuteAsync in a Task and handles cancellation wiring. Most of the time BackgroundService is all you need.

Example

```
// Raw -- manage the task yourself
public class MyService : IHostedService {
public Task StartAsync(CancellationToken ct) { ... }
public Task StopAsync(CancellationToken ct) { ... }
}
// Convenience -- just write the loop
public class MyService : BackgroundService {
protected override Task ExecuteAsync(CancellationToken ct) { ... }
}
```

Q174 What happens if a BackgroundService throws an unhandled exception?

Answer

By default (.NET 6+) the service stops but the host keeps running. Set BackgroundServiceExceptionBehavior to StopHost to crash the whole app and let your process supervisor (Kubernetes, systemd) restart it.

Example

```
builder.Services.Configure<HostOptions>(o =>
o.BackgroundServiceExceptionBehavior =
BackgroundServiceExceptionBehavior.StopHost);
```

Q175 How do you schedule recurring jobs with Quartz.NET and cron?

Answer

Create a class implementing IJob, build a JobDetail and a CronTrigger, schedule them via IScheduler.

Example

```
public class ReportJob : IJob {
public Task Execute(IJobExecutionContext ctx) =>
_report.GenerateAsync(ctx.CancellationToken);
}
var job = JobBuilder.Create<ReportJob>().Build();
var trigger = TriggerBuilder.Create()
.WithCronSchedule("0 0 8 * * ?") // every day at 8 AM
.Build();
```

```
await scheduler.ScheduleJob(job, trigger);
```

Q176 How do you inject services into Quartz.NET jobs?

Answer

Use Quartz.Extensions.DependencyInjection and call UseMicrosoftDependencyInjectionJobFactory. Quartz resolves each job from the DI container so constructor injection works normally.

Example

```
builder.Services.AddQuartz(q => {
q.UseMicrosoftDependencyInjectionJobFactory();
q.AddJob<ReportJob>(opts => opts.WithIdentity("report"));
q.AddTrigger(opts => opts
.ForJob("report")
.WithCronSchedule("0 0 8 * * ?"));
});
builder.Services.AddQuartzHostedService(
q => q.WaitForJobsToComplete = true);
```

Q177 How do you prevent the same job running on multiple instances simultaneously?

Answer

Four options: 1. Quartz.NET clustered AdoJobStore (shared PostgreSQL) -- cluster elects one runner per job. 2. Distributed lock (Redis Redlock or Azure Blob lease) -- first instance to acquire wins. 3. Hangfire with shared database backend -- enqueuing is idempotent. 4. Outbox record with UNIQUE constraint -- only one instance can insert the 'job started' record.

Example

```
var acquired = await _redlock.TryAcquireAsync(
"job:nightly-report", TimeSpan.FromMinutes(10));
if (!acquired) return; // another instance has it
try { await RunReportAsync(); }
finally { await _redlock.ReleaseAsync(); }
```

Q178 How would you architect distributed scheduling across multiple servers?

Answer

Option A (clustered Quartz): shared AdoJobStore in PostgreSQL, cluster coordinates which node runs each job. Automatic failover if leader goes down. Option B (message-driven): a single lightweight scheduler enqueues a message (RabbitMQ, SQS). Worker pods consume and process. Scale workers independently; scheduler stays simple.

Example

```
// Message-driven (recommended for scale):
publisher.Publish(new ProcessExpiredOrdersCommand());

// Any number of worker pods consumes:
consumer.Handler = async msg =>
await _orderService.ExpireAsync(msg);
```

Ch 50 — Event Messaging

Q179 What is MediatR and what pattern does it implement?

Answer

MediatR implements the Mediator and CQRS patterns in-process. A sender posts a request (command or query) to the mediator, which finds and calls the right handler. The sender has no direct dependency on the handler.

Example

```
public record CreateOrderCommand(string Product, int Qty) : IRequest<int>;
public class CreateOrderHandler : IRequestHandler<CreateOrderCommand, int> {
public Task<int> Handle(CreateOrderCommand cmd, CancellationToken ct)
=> _repo.CreateAsync(cmd.Product, cmd.Qty, ct);
}
var orderId = await _mediator.Send(new CreateOrderCommand("Book", 2));
```

Q180 How do you publish notifications with MediatR?

Answer

Use INotification instead of IRequest and call mediator.Publish(). Unlike Send, Publish calls ALL registered handlers -- useful for domain events where multiple things need to react.

Example

```
public record OrderCreated(int OrderId) : INotification;
public class SendConfirmationEmail : INotificationHandler<OrderCreated> {
public Task Handle(OrderCreated n, CancellationToken ct)
=> _mailer.SendAsync(n.OrderId, ct);
}
await _mediator.Publish(new OrderCreated(orderId));
```

Q181 What are the downsides of overusing MediatR?

Answer

Every method call becomes invisible -- F12 takes you to MediatR's Send, not your handler. Pipeline behaviors add indirection. For simple CRUD endpoints it is just extra ceremony. Use it where the decoupling genuinely adds value -- complex CQRS flows, domain events.

Example

```
// Good fit: complex domain with many handlers and cross-cutting concerns
// Overkill: a simple CRUD action that just calls one service method
```

Q182 What is MassTransit?

Answer

MassTransit is a .NET message bus abstraction. Write message contracts and consumer classes; MassTransit handles transport (RabbitMQ, Azure Service Bus, SQS, Kafka), serialisation, consumer routing, retry policies, sagas, and observability.

Example

```
builder.Services.AddMassTransit(x => {
x.AddConsumer<OrderCreatedConsumer>();
x.UsingRabbitMq((ctx, cfg) => {
cfg.Host("rabbitmq://localhost");
cfg.ConfigureEndpoints(ctx);
});
});
```

Q183 How do you configure MassTransit with RabbitMQ?

Answer

Call UsingRabbitMq inside AddMassTransit. ConfigureEndpoints reads your registered consumers and creates the right queues and exchanges automatically.

Example

```
x.UsingRabbitMq((ctx, cfg) => {
cfg.Host(new Uri("amqp://guest:guest@rabbitmq:5672"));
cfg.ConfigureEndpoints(ctx);
});
```

Q184 How do you define and consume messages with MassTransit?

Answer

Declare message contracts as records. Implement IConsumer<TMessage>. MassTransit routes to the right consumer based on the message type name.

Example

```
public record OrderPlaced(Guid OrderId, decimal Amount);
public class OrderPlacedConsumer : IConsumer<OrderPlaced> {
public async Task Consume(ConsumeContext<OrderPlaced> ctx) {
var msg = ctx.Message;
await _warehouse.ReserveAsync(msg.OrderId);
}
}
```

Q185 What advanced features does MassTransit offer?

Answer

Sagas: state machines that persist state across multiple messages -- for long-running workflows. Retry policies: exponential back-off on consumer exceptions, configurable per endpoint. Outbox: writes outgoing message inside the same DB transaction as your domain changes -- guarantees exactly-once publishing.

Example

```
cfg.ReceiveEndpoint("order-placed", e => {
e.UseMessageRetry(r => r.Exponential(
5,
TimeSpan.FromSeconds(1),
TimeSpan.FromMinutes(5),
TimeSpan.FromSeconds(5)));
e.ConfigureConsumer<OrderPlacedConsumer>(ctx);
});
```

BUCKET 4 — SYSTEM DESIGN AND ARCHITECTURE

Ch 51 — APIs, SDKs and Resilience

Q186 What happens if you create a new HttpClient for every API call?

Answer

Each new HttpClient opens its own socket. Disposing it doesn't immediately close the socket -- it lingers in TIME_WAIT for up to 4 minutes. Under load you quickly exhaust available ports (socket exhaustion). DNS changes also go undetected because each client has its own DNS cache.

Example

```
// Bad -- socket exhaustion under load
for (int i = 0; i < 1000; i++) {
using var client = new HttpClient();
await client.GetAsync("https://api.example.com/data");
}
// Good -- reuse via IHttpClientFactory
public class MyService(HttpClient client) { }
```

Q187 What is HttpClientFactory and why was it introduced?

Answer

HttpClientFactory manages a pool of HttpMessageHandler instances (default 2-minute lifetime). Handlers are recycled to respect DNS changes while sockets are reused. Integrates cleanly with DI and Polly.

Example

```
builder.Services.AddHttpClient("PaymentsApi", c =>
c.BaseAddress = new Uri("https://payments.example.com"));
```

Q188 What is a typed HttpClient and how does it differ from a named client?

Answer

Named client: inject IHttpClientFactory, call CreateClient("name") -- loose coupling but magic strings. Typed client: inject a class that wraps HttpClient -- strongly typed, mockable in tests, no magic strings.

Example

```
public class PaymentsClient {
private readonly HttpClient _http;
public PaymentsClient(HttpClient http) => _http = http;
public Task<PaymentResult> ChargeAsync(ChargeDto dto)
=> _http.PostAsJsonAsync("charges", dto).ContinueWith(/* parse */);
}
builder.Services.AddHttpClient<PaymentsClient>(c =>
c.BaseAddress = new Uri("https://payments.example.com"));
```

Q189 What is Refit?

Answer

Refit generates HttpClient implementations from annotated C# interfaces. You define the API contract with attributes; Refit generates the HTTP calls, URL building, and serialisation.

Example

```
public interface IPaymentsApi {
[Post("/charges")]
Task<PaymentResult> ChargeAsync([Body] ChargeDto dto);
```

```
[Get("/charges/{id}")]
Task<PaymentResult> GetAsync(string id);
}
builder.Services.AddRefitClient<IPaymentsApi>()
.ConfigureHttpClient(c =>
c.BaseAddress = new Uri("https://payments.example.com"));
```

Q190 What is a DelegatingHandler and what is it used for?

Answer

DelegatingHandler is middleware for the HttpClient pipeline. Override SendAsync, modify the request or response, call the inner handler. Common uses: injecting auth tokens, logging, adding correlation IDs, implementing retry logic.

Example

```
public class AuthHandler : DelegatingHandler {
protected override async Task<HttpResponseMessage> SendAsync(
HttpRequestMessage req, CancellationToken ct)
{
req.Headers.Authorization = new AuthenticationHeaderValue(
"Bearer", await GetTokenAsync());
return await base.SendAsync(req, ct);
}
}
```

Q191 What is Polly and how do you use it with HttpClientFactory?

Answer

Polly is a resilience library with policies for retry, circuit breaker, timeout, bulkhead, and fallback. Hook it into HttpClientFactory with AddPolicyHandler -- the policy wraps every outgoing request from that client.

Example

```
builder.Services.AddHttpClient<PaymentsClient>()
.AddPolicyHandler(HttpPolicyExtensions
.HandleTransientHttpError()
.RetryAsync(3));
```

Q192 How do you add an exponential back-off retry policy with Polly?

Answer

Use HandleTransientHttpError (covers 5xx and network errors) and WaitAndRetryAsync with a back-off formula.

Example

```
var retry = HttpPolicyExtensions
.HandleTransientHttpError()
.WaitAndRetryAsync(
retryCount: 3,
sleepDurationProvider: attempt =>
TimeSpan.FromSeconds(Math.Pow(2, attempt)));

builder.Services.AddHttpClient<OrdersClient>()
.AddPolicyHandler(retry);
```

Q193 How do you implement a circuit breaker with Polly?

Answer

CircuitBreakerAsync opens the circuit after N consecutive failures, fast-failing all requests for a break duration. After the break it lets one probe request through (half-open). Prevents hammering a failing downstream service.

Example

```
var cb = HttpPolicyExtensions
.HandleTransientHttpError()
.CircuitBreakerAsync(
handledEventsAllowedBeforeBreaking: 5,
durationOfBreak: TimeSpan.FromSeconds(30));

builder.Services.AddHttpClient<OrdersClient>()
.AddPolicyHandler(cb);
```

Q194 How do you implement a fallback strategy with Polly?

Answer

FallbackAsync catches a failure and returns a pre-defined fallback response. Combine with circuit breaker and retry for a full resilience stack.

Example

```
var fallback = Policy<HttpResponseMessage>
.HandleResult(r => !r.IsSuccessStatusCode)
.FallbackAsync(_ => {
var res = new HttpResponseMessage(HttpStatusCode.OK);
res.Content = new StringContent("{"source":"cache"}");
return Task.FromResult(res);
});
```

Q195 How do you handle token refresh for outgoing HTTP requests?

Answer

Use a DelegatingHandler that fetches a cached token and adds it as a Bearer header. On a 401 response, invalidate the cache, get a fresh token, and retry once.

Example

```
public class BearerTokenHandler : DelegatingHandler {
protected override async Task<HttpResponseMessage> SendAsync(
HttpRequestMessage req, CancellationToken ct)
{
var token = await _cache.GetOrRefreshAsync(ct);
req.Headers.Authorization = new("Bearer", token);
var resp = await base.SendAsync(req, ct);
if (resp.StatusCode == HttpStatusCode.Unauthorized) {
_cache.Invalidate();
token = await _cache.GetOrRefreshAsync(ct);
req.Headers.Authorization = new("Bearer", token);
resp = await base.SendAsync(req, ct);
}
return resp;
}
}
```

Ch 52 — OpenTelemetry & Observability

Q196 What are the three pillars of observability in OpenTelemetry?

Answer

Traces -- distributed request flows across services, composed of spans linked by a trace ID. Metrics -- numeric measurements over time: request rate, latency percentiles, error rate. Logs -- structured event records correlated to traces via the trace ID.

Example

```
builder.Services.AddOpenTelemetry()
.WithTracing(t =>
t.AddAspNetCoreInstrumentation().AddOtlpExporter())
.WithMetrics(m =>
m.AddAspNetCoreInstrumentation().AddOtlpExporter())
.WithLogging(l => l.AddOtlpExporter());
```

Q197 What is the OpenTelemetry Collector?

Answer

The Collector is a vendor-neutral agent that sits between your apps and your observability backends. Apps send telemetry in standard OTLP format. The Collector filters, enriches, and fans out to Jaeger, Prometheus, Datadog, etc. -- without your app code needing to know which backends you use.

Example

```
// App sends to Collector:
o.Endpoint = new Uri("http://otel-collector:4317");

// otel-config.yaml:
// receivers: [otlp]
// exporters: [jaeger, prometheus]
// pipelines: traces->[jaeger], metrics->[prometheus]
```

Q198 What is a span and what is a trace?

Answer

A trace is the complete end-to-end journey of one request through all services, identified by a trace ID. A span is one unit of work within that trace -- an HTTP call, a DB query. Spans have a start time, duration, parent span ID, and arbitrary attributes, forming a tree.

Example

```
using var activity = _tracer.StartActivity("ProcessOrder");
activity?.SetTag("order.id", orderId);
// Any async calls inside create child spans automatically
```

Q199 How does OTel propagate context between microservices?

Answer

ASP.NET Core's OTel instrumentation automatically propagates W3C TraceContext headers (traceparent, tracestate) on outgoing HttpClient calls and reads them from incoming requests. Baggage carries custom key-value data along the same chain.

Example

```
Baggage.SetBaggage("tenantId", tenantId);
// Included in all downstream HttpClient calls automatically
var tid = Baggage.GetBaggage("tenantId");
```

Q200 What are OTel sampling strategies?

Answer

AlwaysOn -- record everything. Only feasible for low-traffic. TraceIdRatioBased -- record a fixed percentage (e.g., 10%). ParentBased -- inherit parent's sampling decision, keeping complete traces. Tail sampling -- decide after the fact, keeping all error traces. Requires the Collector.

Example

```
builder.Services.AddOpenTelemetry()
.WithTracing(t => t
.SetSampler(new TraceIdRatioBasedSampler(0.1))
```

```
.AddAspNetCoreInstrumentation());
```

Q201 How do you create custom metrics in .NET?

Answer

Use System.Diagnostics.Metrics.Meter to create counters, histograms, and gauges. OTel picks them up via AddMeter when you configure the metrics pipeline.

Example

```
public class OrderMetrics {
private readonly Counter<int> _created;
private readonly Histogram<double> _total;
public OrderMetrics(IMeterFactory factory) {
var m = factory.Create("Orders");
_created = m.CreateCounter<int>("orders.created");
_total = m.CreateHistogram<double>("orders.total", "USD");
}
public void RecordOrder(decimal total) {
_created.Add(1);
_total.Record((double)total);
}
}
```

Q202 What are trace links in OTel and when are they useful?

Answer

Trace links associate a span with spans from other traces. Classic use: a batch job links to each individual request's trace it is processing, showing causal relationships without merging them into one giant trace.

Example

```
var links = incomingMessages
.Select(m => new ActivityLink(m.TraceContext))
.ToList();
using var activity = source.StartActivity(
"ProcessBatch", ActivityKind.Internal,
parentContext: default, links: links);
```

Q203 How do you handle high-cardinality data in metrics?

Answer

High cardinality (userId, request path with IDs as tags) creates a new time series per unique value -- can crash your metrics backend. Use Views to allowlist only low-cardinality tags you actually need.

Example

```
builder.Services.AddOpenTelemetry()
.WithMetrics(m => m
.AddView("orders.total",
new MetricStreamConfiguration {
TagKeys = new[] { "region", "status" }
// userId and request.path dropped
}));
```

Ch 54 — Build, Deploy & Distributed Systems

Q204 What is the difference between framework-dependent and self-contained deployment?

Answer

Framework-dependent: the .NET runtime must be installed on the target machine. Small output, easy to patch the runtime separately. Self-contained: runtime is bundled with the app. Larger but runs anywhere with no .NET pre-installed.

Example

```
dotnet publish -c Release # framework-dependent
dotnet publish -c Release -r linux-x64 --self-contained # self-contained
```

Q205 How do you publish for a specific platform?

Answer

Use -r with a Runtime Identifier (RID). Common: win-x64, linux-x64, linux-arm64, osx-arm64.

Example

```
dotnet publish -c Release -r linux-x64 --self-contained -o ./publish

# Single-file:
dotnet publish -c Release -r linux-x64 -p:PublishSingleFile=true --self-contained
```

Q206 How do you manage environment-specific config at deployment?

Answer

ASP.NET Core loads appsettings.json then overlays appsettings.{ASPNETCORE_ENVIRONMENT}.json. Set the environment variable at deploy time. Sensitive values should come from environment variables or a secrets manager -- never appsettings files committed to source control.

Example

```
appsettings.json # base
appsettings.Production.json # prod overrides
appsettings.Staging.json # staging overrides
# Set at runtime:
# ASPNETCORE_ENVIRONMENT=Production
```

Q207 How do you write a Dockerfile for an ASP.NET Core app?

Answer

Use a multi-stage build: compile in the SDK image, copy only the published output into the lean runtime image. Final image has no SDK, no source, no build tools.

Example

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:9.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "MyApp.dll"]
```

Q208 What is a multi-stage Docker build and why use it?

Answer

Multiple FROM instructions. Compile in a large SDK image, copy only the output into a lean runtime image. Final image has no compiler, no source, no NuGet cache -- typically 3-5x smaller and much smaller attack surface.

Example

```
# Stage 1 -- build
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
```

```
RUN dotnet publish -c Release -o /app

# Stage 2 -- runtime only
FROM mcr.microsoft.com/dotnet/aspnet:9.0
COPY --from=build /app /app
ENTRYPOINT ["dotnet", "/app/MyApp.dll"]
```

Q209 How do you minimise Docker image size for .NET apps?

Answer

Multi-stage build + runtime base image. Enable PublishTrimmed to remove unused assemblies. PublishSingleFile packs everything into one binary. Native AOT produces the smallest result of all -- a standalone native binary.

Example

```
dotnet publish -c Release -r linux-x64 \
-p:PublishTrimmed=true \
-p:PublishSingleFile=true \
--self-contained
```

Q210 How do you pass config and secrets to a Docker container?

Answer

Use -e flags on docker run, the environment section in Docker Compose, or ENV in the Dockerfile for non-secret defaults. For secrets use Docker Secrets or mount from a secrets manager -- never bake them into the image.

Example

```
docker run \
-e ASPNETCORE_ENVIRONMENT=Production \
-e ConnectionStrings__DefaultConnection="Host=db;" \
myapp
```

Q211 How do you write a Docker Compose file for ASP.NET Core + PostgreSQL + Redis?

Answer

Define three services. Use depends_on so the app doesn't start before the database. Pass connection strings as environment variables.

Example

```
services:
app:
image: myapp
ports: ["8080:8080"]
environment:
- ConnectionStrings__Db=Host=postgres;Database=app;Username=app;Password=secret
- ConnectionStrings__Redis=redis:6379
depends_on: [postgres, redis]
postgres:
image: postgres:16
environment: { POSTGRES_PASSWORD: secret }
redis:
image: redis:7
```

Q212 How do you use health checks in Docker Compose?

Answer

Add a healthcheck block to the service. Docker polls it and marks the container healthy or unhealthy. Use condition: service_healthy in depends_on to make the app wait until the DB is actually ready.

Example

```
postgres:
image: postgres:16
healthcheck:
test: ["CMD-SHELL", "pg_isready -U app"]
interval: 5s
retries: 5
app:
depends_on:
postgres:
condition: service_healthy
```

Q213 What is .NET Aspire?

Answer

.NET Aspire is an opinionated stack for cloud-native apps. Instead of writing Docker Compose or Helm charts, you declare your services, databases, caches, and message buses in a C# AppHost project. Aspire starts all containers locally, provides a built-in dashboard with distributed traces and metrics, and handles service discovery wiring.

Example

```
var builder = DistributedApplication.CreateBuilder(args);
var db = builder.AddPostgres("db");
var cache = builder.AddRedis("cache");
builder.AddProject<Projects.OrdersApi>("orders-api")
.WithReference(db)
.WithReference(cache);
builder.Build().Run();
```

Q214 How do you deploy an Aspire application to Docker Compose?

Answer

Run the Aspire publish command to generate a docker-compose.yaml from the AppHost definition, then deploy normally.

Example

```
dotnet aspire publish --format docker-compose
docker compose -f docker-compose.yaml up -d
```

Q215 How do you configure OpenTelemetry in an Aspire application?

Answer

Call builder.AddServiceDefaults() in each service project. Aspire injects OTEL_EXPORTER_OTLP_ENDPOINT automatically. ServiceDefaults wires up traces, metrics, and logs pointing at Aspire's built-in dashboard.

Example

```
// In each service's Program.cs:
builder.AddServiceDefaults();
// registers OTel, health checks, service discovery
// Aspire sets OTEL_EXPORTER_OTLP_ENDPOINT automatically
```